VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi – 590018, Karnataka.

A Project Report
on

"PharmaCompare: A Real-Time Drug Price and Prescription Intelligence Platform"

Submitted in partial fulfillment for the award of degree of

BACHELOR OF ENGINEERING

in

Department of Information Science & Engineering

Project Associates

1. Pramod S D	4GM23IS031
2. Rajesh R G	4GM23IS036
3. Vaibhav R Katagi	4GM23IS053
4. Vanshika D Jain	4GM23IS055

Academic Year 2026-27

Under the Guidance of

Dr. Neelambike S
B.E., M.Tech., Ph.D.,
Professor and Head

PROJECT CO-ORDINATOR
Mrs. Niveditha N M
B.E.,M.Tech.(Ph.D).,

HEAD OF THE DEPARTMENT
Dr. Neelambike S
B.E.,M.Tech, Ph.D.,

Srishyla Educational Trust (R), Bheemasamudra
GM INSTITUTE OF TECHNOLOGY, DAVANGERE
Approved by AICTE | Affiliated by V.T.U Belagavi | Recognized by Govt. of Karnataka

Srishyla Educational Trust (R), Bheemasamudra

GM INSTITUTE OF TECHNOLOGY, DAVANGERE

Approved by AICTE | Affiliated by V.T.U Belagavi | Recognized by Govt. of Karnataka

Department of Information Science and Engineering

CERTIFICATE

Certified that the Project Report entitled "**PharmaCompare: A Real-Time Drug Price and Prescription Intelligence Platform**" carried out by **Pramod S D (4GM23IS031), Rajesh R G (4GM23IS036), Vaibhav R Katagi (4GM23IS053), Vanshika D Jain (4GM23IS055),** bonafide students of VII Semester in partial fulfillment for the award of Bachelor of Engineering in **Department of Information Science & Engineering**, GM Institute of Technology, Visvesvaraya Technological University, Belagavi for the academic year 2026-27. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Dr. Neelambike S Project Guide	Dr. Neelambike S Head of the Department	Dr. K S Onkarappa Principal

EXAMINER:

INTERNAL	EXTERNAL
Name:	
Signature:	

ABSTRACT

Reading a prescription, checking its medicines for safety problems and finding cheaper comparable products are normally three separate manual tasks. This project presents **PharmaCompare**, a web platform that joins them. A user uploads a prescription image or PDF; the text is read with **Google Cloud Vision or EasyOCR**, grouped into lines and matched against a reference table of **253,973 Indian medicine products**. A line is accepted automatically only with strong evidence; every other line is held as **"needs verification"** until the user confirms it, with the reasons shown, a warning if the choice differs from the prescription and a recorded audit trail. The dosing written on the prescription is read into a **medicine timeline**. Confirmed medicines are screened for **drug interactions (DDInter 2.0), duplicate ingredients and allergies**, and are priced against products with the **same composition, strength, form and pack quantity**. A photo of a medicine pack can be scanned for possible products, and a receiver for pharmacy price and stock reports is built, but **no pharmacy feed is connected**. The system uses **React, Flask and MySQL** and is rule-based, with no trained machine-learning model; prices come from a static dataset, not from live pharmacies. All **592 back-end and 186 front-end tests passed**. Evaluation on real prescriptions remains future work.

ACKNOWLEDGEMENT

We are grateful to **Dr. K S Onkarappa**, the **Principal of G M Institute of Technology, Davanagere**, for his unwavering support and commitment to excellence in education. His guidance has inspired us to strive for greater heights.

We extend our appreciation to **Dr. Neelambike S**, the **Head of the Department of Information Science and Engineering**, for her encouragement and support throughout this project. Her leadership and vision have been instrumental in fostering an environment conducive to academic growth.

We would like to express our appreciation to our project coordinator **Mrs. Niveditha N M, Assistant Professor**, for her invaluable guidance and support. Her expertise has been indispensable in shaping our work.

We would like to express our deep appreciation to our project guide **Dr. Neelambike S, Professor and Head**, for her invaluable guidance, support and constructive feedback. Her expertise has been indispensable in shaping our work.

We are also thankful to the teaching and non-teaching staff of the **Department of Information Science and Engineering, G M Institute of Technology** for their dedication and assistance during our academic journey. We also acknowledge the support of our classmates and friends for their encouragement throughout this project.

Last but not least, we express our gratitude to our families for their understanding, patience and unwavering support during our academic pursuits. Their love and encouragement have been the cornerstone of our success.

Pramod S D	**4GM23IS031**
Rajesh R G	**4GM23IS036**
Vaibhav R Katagi	**4GM23IS053**
Vanshika D Jain	**4GM23IS055**

CONTENTS

CHAPTER NO.	CHAPTERS	PAGE NO.
	ABSTRACT	**i**
	ACKNOWLEDGEMENT	**ii**
	CONTENTS	**iii**
	LIST OF FIGURES	**v**
	LIST OF TABLES	**vi**
1	**PREAMBLE**	**1**
	1.1 PROBLEM STATEMENT	1
	1.2 THE SOLUTION	1
	1.3 OBJECTIVES OF PROJECT	2
	1.4 ADVANTAGES	2
	1.5 LITERATURE SURVEY	3
	1.6 ORGANIZATION OF REPORT	6
2	**SOFTWARE REQUIREMENT SPECIFICATION**	**10**
	2.1 FUNCTIONAL OVERVIEW	10
	2.2 USER CHARACTERISTICS	11
	2.3 INPUT REQUIREMENTS	11
	2.3.1 USER INPUT REQUIREMENTS	12
	2.3.2 REFERENCE DATA REQUIREMENTS	12
	2.4 OUTPUT REQUIREMENTS	13
	2.4.1 SCREEN AND REPORT OUTPUT	13
	2.4.2 SAFETY ALERTS AND NOTICES	13
	2.5 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS	14
	2.5.1 FUNCTIONAL REQUIREMENTS	14
	2.5.2 NON-FUNCTIONAL REQUIREMENTS	14
	2.6 SOFTWARE REQUIREMENTS	15
	2.7 HARDWARE REQUIREMENTS	15
	2.8 PROJECT CYCLE	16

CHAPTER NO.	CHAPTERS	PAGE NO.
3	**SYSTEM DESIGN**	**17**
	3.1 PROJECT ARCHITECTURE	17
	3.2 DATABASE DESIGN	19
	3.3 SEQUENCE DIAGRAM	21
	3.4 DATA FLOW DIAGRAM	22
	3.5 DATABASE TABLE STRUCTURE	24
	3.6 FLOW CHART	32
	3.7 UTILITIES	33
4	**IMPLEMENTATION**	**34**
	4.1 SOFTWARE TOOLS USED	34
	4.2 IMPLEMENTATION DETAILS	34
	4.3 FRONT-END FORMS DESIGN	42
5	**TESTING**	**48**
	5.1 TESTING OBJECTIVES	48
	5.2 SCOPE	48
	5.3 UNIT TESTING	49
	5.4 MODULE TESTING	51
	5.5 FUNCTIONAL TESTING	52
	5.6 SYSTEM TESTING	54
6	**RESULTS**	**56**
	6.1 SNAPSHOTS OF THE PROJECT	56
	6.2 OBSERVATIONS	60
7	**CONCLUSION**	**64**
	7.1 CONCLUSION	64
	7.2 FUTURE SCOPE	65
	APPENDIX-A	**66**
	APPENDIX-B	**67**
	REFERENCES	**68**

LIST OF FIGURES

FIGURE NO.	DESCRIPTION	PAGE NO.
2.1	PROJECT CYCLE	16
3.1	PROJECT ARCHITECTURE	18
3.2	ENTITY-RELATIONSHIP DIAGRAM	20
3.3	SEQUENCE DIAGRAM	22
3.4	DATA FLOW DIAGRAM, LEVEL 0	23
3.5	DATA FLOW DIAGRAM, LEVEL 1	24
3.6	FLOW CHART	32
4.1	MEDICINE MATCHING AND MATCH STATUSES	37
4.2	LANDING PAGE	42
4.3	REGISTRATION FORM	43
4.4	LOGIN FORM	43
4.5	UPLOAD PAGE	44
4.6	VERIFICATION DIALOG	44
4.7	WARNING BEFORE A CONFLICTING CONFIRMATION	45
4.8	DASHBOARD	45
4.9	MEDICINE SEARCH PAGE	46
4.10	SCAN A MEDICINE PAGE	46
4.11	DATA SOURCES ON THE ABOUT PAGE	47
6.1	FICTIONAL DEMONSTRATION PRESCRIPTION	56
6.2	ANALYSIS SUMMARY	56
6.3	DETECTED MEDICINES	57
6.4	DETAILS OF A LINE THAT NEEDS VERIFICATION	58
6.5	MEDICINE TIMELINE	58
6.6	SAFETY AND INTERACTIONS	59
6.7	SMART HEALTH REPORT (PAGES 1 AND 2)	59
6.8	WHY INGREDIENT-ONLY SUGGESTIONS FAIL	61
6.9	MATCHER OUTCOMES BY SIMULATED EDITS	62

LIST OF TABLES

TABLE NO.	DESCRIPTION	PAGE NO.
2.1	USER INPUT REQUIREMENTS	12
2.2	REFERENCE DATA AND SERVICE REQUIREMENTS	12
2.3	SCREEN AND REPORT OUTPUT	13
2.4	SOFTWARE REQUIREMENTS	15
2.5	HARDWARE REQUIREMENTS	15
3.1	DATABASE TABLES AND THEIR PURPOSE	19
3.2	STRUCTURE OF THE DATABASE TABLES	25
3.3	UTILITY MODULES OF THE BACK END	33
4.1	SOFTWARE TOOLS USED	34
4.2	REST API ENDPOINTS	40
4.3	THRESHOLDS AND PARAMETERS	41
5.1	UNIT TESTS	49
5.2	MODULE TESTS	51
5.3	FUNCTIONAL TESTS	53
5.4	BACK-END TEST SUITE BY GROUP	54
5.5	FRONT-END LOGIC TEST SUITE BY GROUP	54
6.1	REFERENCE DATA USED BY THE SYSTEM	60
6.2	INGREDIENT-ONLY VERSUS COMPARABLE RULE	61
6.3	MATCHER BEHAVIOUR UNDER SIMULATED NOISE	62
6.4	STATUS OF THE OBJECTIVES	63

CHAPTER 1

PREAMBLE

The preamble comprises of problem statement, solution, objectives, advantages, literature survey and organization of the report.

1.1 PROBLEM STATEMENT

A prescription is useful only if the patient can tell what was prescribed. In one South African district hospital, pharmacists read only 75% of sampled handwritten prescriptions correctly, and one in five health workers read a 4 mg lorazepam injection as 40 mg [1]. A wrongly read medicine carries through to a wrong safety message and a wrong price.

A prescription with several medicines also risks duplicate therapy, interactions and allergy conflicts that a patient cannot easily check, and products with the same composition are sold at very different prices [2], [3]. There is a pressing need for a system that takes a user from a prescription image to a checked list of medicines and comparable prices without presenting a guess as a fact.

1.2 THE SOLUTION

The proposed system, **"PharmaCompare"**, is a web application that joins prescription reading, medicine identification, safety screening and price comparison in one workflow. The user uploads a photograph or PDF of a prescription and receives a **Smart Health Report**.

The text is read with Google Cloud Vision or the local EasyOCR engine and matched against 253,973 Indian medicine products. A line is accepted automatically only with strong evidence; every other line is held as *needs verification* until the user confirms it. Confirmed medicines are screened against 160,235 documented interaction pairs (DDInter 2.0) and priced against products with the same ingredients, strength, dosage form, release type and pack quantity. The dosing written on the prescription is read into a **medicine timeline**, and a photo of a medicine pack can be scanned to find possible products.

Prices come from a **static community dataset**, not from live pharmacies. The system can receive prices and stock that a pharmacy reports and shows them separately, but **no pharmacy feed is connected**, so no live price or stock level is known.

1.3 OBJECTIVES OF PROJECT

The main objective of this project is to **design and develop PharmaCompare**, a platform that helps a patient understand a prescription, check it for safety problems and find cheaper comparable products. The specific objectives are:

1. To develop a **real-time drug price comparison module** for products of the same composition, strength and form.
2. To implement an **AI-based prescription analysis system** that checks medicines for interactions, duplicates and allergies.
3. To design an **alternative drug recommendation engine** that suggests cheaper comparable products, with reasons.
4. To integrate **real-time drug availability tracking**.
5. To incorporate **confidence-based decision support** that hands uncertain readings back to the user.
6. To utilise and process **healthcare data sources** as the knowledge base of the system.

1.4 ADVANTAGES

The adoption of PharmaCompare offers several advantages over reading and checking a prescription by hand. Key advantages include:

1. **One Workflow**: reading, safety screening, price comparison and the Smart Health Report in a single upload.
2. **Cautious by Design**: a guessed line never reaches the safety or price stages until the user confirms it, and a missing interaction record is never shown as “safe”.
3. **Medicine Timeline**: the written dosing is shown as a morning, afternoon, evening and night schedule.
4. **Strict Price Comparison**: same ingredients, strength, form and release type, with savings per unit and the source of every price stated.
5. **Security and Privacy**: salted password hashes, owner-only access and a fully local OCR option.
6. **Low Cost and Extensible**: free open data, an open-source stack and separate service modules.

1.5 LITERATURE SURVEY

[1]. H. Brits et al. “Illegible handwriting and other prescription errors on prescriptions at National District Hospital, Bloemfontein” (South African Family Practice, 2017). Twenty doctors’ prescriptions were read by doctors, nurses and pharmacists. Doctors read 88%, nurses 82% and pharmacists 75% correctly, and 20% of health workers read lorazepam 4 mg as 40 mg. Limitation: one hospital and a small sample.

[2]. A. Ray et al. “A cost variation analysis of drugs available in the Indian market for the management of thromboembolic disorders” (Cureus, 2020). Compares brand prices of anticoagulant, antiplatelet and fibrinolytic drugs per tablet, capsule or injection and reports wide cost variation between brands. Limitation: catalogue prices for one drug group.

[3]. D. K. Behera et al. “Medicine affordability and access in India: lessons from generic–branded price variation under the Jan Aushadhi Scheme” (Frontiers in Public Health, 2025). Analyses 696 branded eye and ENT formulations against Jan Aushadhi generics and finds wide price variation among brands. Limitation: one therapeutic area.

[4]. A. Razdan et al. “Recognition of handwritten medical prescription using CNN Bi-LSTM with lexicon search” (ICCCNT, 2023). A CNN–BiLSTM recogniser whose predictions are checked by a lexicon search of Indian drug names; the lexicon helps on prescriptions. Limitation: reading only; no price or safety.

[5]. M. F. Uz Zaman et al. “Recognizing medicine names from Bangladeshi handwritten prescription images using TrOCR” (CEII, 2024). TrOCR fine-tuned on 893 prescription images, followed by edit-distance correction: character error rate falls from 16% to 7% and word error rate from 36% to 11%. Limitation: one country’s dataset.

[6]. U. Ali et al. “Leveraging deep learning with multi-head attention for accurate extraction of medicine from handwritten prescriptions” (ICRAI, 2024). Mask R-CNN finds medicine regions, TrOCR reads them and the text is matched to a database; 1.4% character error rate on standard benchmarks. Limitation: the benchmark is not the authors’ own dataset.

[7]. A. Roy et al. “A YOLO-TrOCR pipeline with dictionary refinement for handwritten medicine name extraction” (PECCII, 2026). YOLO detects medicine names, TrOCR reads them and a dictionary refines the output, raising exact match from 0.7355 to 0.7518. Limitation: Bangladesh-specific dictionary.

[8]. M. T. S. Devi et al. “GENERIC RX: AI-powered prescription OCR for generic medicine discovery” (ICICDS, 2026). Prescription OCR with fuzzy matching, cheaper generic substitutes, interaction screening and cost comparison, tested on 500 real prescription images. Limitation: only the abstract was read; comparability rules are unknown.

[9]. S. Thota et al. “AI-powered prescription decoder and generic medicine recommender” (International Journal on Science and Technology, 2026). Reads prescriptions, maps medicines to RxNorm for generic equivalents and shows prices and warnings; unrecognised medicines are flagged for manual review. Limitation: no quantitative evaluation.

[10]. E. Diaz Amaya et al. “Information system for price comparison and prescription reading … for pharmacies in Peru” (LACCEI, 2024). Scrapes pharmacy websites for prices and reads prescriptions with Pytesseract; on 20 prescriptions it reached 90.5% scanning accuracy. Limitation: a tiny test; live scraping is not done in this project.

[11]. S. Jeevidha et al. “An advanced electronic health record for prescription analysis using YOLO and OCR with spell correction” (ICCIDS, 2026). YOLO finds fields, OCR reads them and spell correction repairs the text (mAP@0.5 of 0.93). Limitation: detection metrics only.

[12]. T. Mankash et al. “MIRAGE: multimodal identification and recognition of annotations in Indian general prescriptions” (arXiv:2410.09729, 2024). Multimodal language models fine-tuned on 743,118 simulated prescriptions reach 82% accuracy. Limitation: simulated data; a preprint.

[13]. M. Agrawal et al. “A review of artificial intelligence in medical prescription analysis” (ICPCSN, 2023). Reviews AI for medication identification, dose calculation and interaction checks, and concludes that accuracy still needs improvement.

[14]. R. Ma et al. “An intelligent OCR-LLM prescription review system for inpatient narcotic and psychotropic drugs” (Medicine, 2026). OCR, a local language model and rules review prescriptions; on 500 prescriptions the system reached 95.6% accuracy (kappa 0.897) against pharmacist review. Limitation: one drug class.

[15]. R. Patil et al. “Application of fuzzy matching algorithms for doctors handwriting recognition” (IBSSC, 2022). A CRNN–CTC reader with edit-distance correction; Damerau–Levenshtein distance worked best on 600 images and a list of only 50 medicines.

[16]. W. T. Kim et al. “Medication extraction and drug interaction chatbot” (Mayo Clinic Proceedings: Digital Health, 2024). OCR of Korean medicine packages, text-similarity matching (threshold 0.7) and a language model with a contraindication database; 99.17% word accuracy on 150 samples. Limitation: mock-up packages, not real prescriptions.

[17]. A. Jagatap et al. “RxLens: multi-agent LLM-powered scan and order for pharmacy” (NAACL, 2025). A multi-agent system reads prescriptions and builds a pharmacy cart by catalogue matching; recall@3 improved by 19–40% for handwriting. Limitation: no safety check or price comparison.

[18]. S. Abdullah et al. “An offline smartphone check for drug interactions and past reactions in handwritten prescriptions” (arXiv:2608.27239, 2026). Each handwritten line is accepted, sent for confirmation or sent for a manual check before documented interactions are retrieved; 62% of lines still needed a manual check in simulation. Limitation: a preprint; no price analysis. It is the closest match to the confirmation gate used here, so that idea is not claimed as new.

[19]. Y. Tian et al. “DDInter 2.0: an enhanced drug interaction resource” (Nucleic Acids Research, 2025). Release 2.0 reports 2,310 drugs and 302,516 interaction records. It is the interaction source of this project; the bulk files that could be downloaded gave 160,235 unique pairs.

[20]. O. M. Alharbi “Designing an OCR-based medication support system with drug–drug interaction alerts for elderly” (IEEE Access, 2026). An application for Arabic-speaking elderly users: OCR identifies the medicines and interactions are flagged in Arabic; a formative usability test was positive. Limitation: no accuracy figures; no price.

[21]. F. Nasser et al. “PharmaBot: a multimodal AI system for drug interaction detection” (IMCET, 2026). Detection, OCR, a language model and retrieval over DrugBank; F1 of 0.9913 for end-to-end interaction detection. Limitation: evaluated on author-made template data.

[22]. K. Gaikwad et al. “OCR and NER based generic medicine recommendation system” (CISES, 2025). Four OCR engines, named-entity recognition and fuzzy matching to over 21,700 brand–generic mappings; over 98% accuracy is claimed. Limitation: the test set is not defined.

[23]. M. Janani et al. “OCR text recognition and recommendation using machine learning” (ICVADV, 2025). OCR plus fuzzy matching of ingredient names and recommendation of pharmaceutical and Ayurvedic alternatives. Limitation: price handling is not described.

[24]. J. Karmakar et al. “Optimizing pharmaceutical affordability: a composition-based algorithm for medicine price and alternative discovery” (ICoEIT, 2025). Ranks low-priced substitutes by ingredient, cost, manufacturer credibility and packaging over about 2.5 lakh Indian medicines; savings above 60% in case studies. Limitation: no prescription input.

[25]. J. S. Larson M. et al. “AI-enabled brand-to-generic medication recommendation system for pharmacy” (IC3ET, 2026). A chatbot that tells brand from generic queries and suggests cheaper options by active ingredient; over 80% accuracy on straightforward queries. Limitation: text queries only.

The above literature reveals significant research interest in each part of the problem, but mostly in separate parts. Handwriting recognition [4]–[7], OCR with interaction alerts [20], [21] and prescription-to-generic or prescription-to-price pipelines [8]–[10], [22]–[24] already exist, and an accept / confirm / manual-check policy has been described [18]. PharmaCompare therefore does not claim a new pipeline: it combines a strict comparability rule, the exclusion of unconfirmed readings from the safety and price stages, and an evaluation of the comparison step. This describes the papers read, not all prior work.

1.6 ORGANIZATION OF REPORT

Chapter 1: Preamble

The preamble in a chapter provides a concise introduction, outlining the chapter's main themes and objectives. It may include background information and context to help readers understand the subject matter. Additionally, the preamble serves as a roadmap, guiding readers through the upcoming content.

This section includes:

1. Problem statement
2. The solution
3. Objectives of the project

4. Advantages of the project
5. Literature survey

Chapter 2: Software Requirement Specification

A Software Requirements Specification (SRS) is a concise document. It outlines the functional and non-functional requirements of a software system, providing a blueprint for development and ensuring alignment on project goals and constraints.

This section includes:

1. Functional overview
2. User characteristics
3. Input requirements
4. Output requirements
5. Functional and non-functional requirements
6. Software requirements
7. Hardware requirements
8. Project cycle

Chapter 3: System Design

System design involves the process of creating a detailed blueprint for a software system, including its architecture, components, interfaces, and interactions. It encompasses defining system requirements, designing data structures, algorithms, and interfaces, and making decisions about technologies, platforms, and frameworks to be used.

This section includes:

1. Project architecture
2. Database design
3. Sequence diagram
4. Dataflow diagram
5. Database table structure
6. Flow chart
7. Utilities

Chapter 4: Implementation

Implementation refers to the process of translating a design or plan into a working software system. It involves writing code, integrating components, and testing the system to ensure that it functions according to the specified requirements.

Implementation typically follows the system design phase and involves programming, debugging, and optimizing the software to meet performance, reliability, and usability goals. Successful implementation results in a fully functional software product ready for deployment and use by end-users.

This section includes:

1. Software tools used
2. Implementation details
3. Front end forms design

Chapter 5: Testing

Testing is the process of evaluating a software system to identify defects, errors, or discrepancies between expected and actual outcomes. It involves executing test cases, scripts, or scenarios to verify that the software functions correctly and meets specified requirements. Testing encompasses various techniques such as unit testing, integration testing, system testing, and acceptance testing, aiming to ensure the quality, reliability, and robustness of the software.

This section includes:

1. Testing objectives
2. Scope
3. Unit testing
4. Module testing
5. Functional testing
6. System testing

Chapter 6: Results

The results section summarizes and presents the findings of the study.

This section includes:

1. Snapshots of the project
2. Observations

Chapter 7: Conclusion

The conclusion section summarizes the main findings and outcomes of the project, offering insights into its achievements, challenges, and potential future directions. It provides closure to the report and highlights the project's contributions to the field of software development.

This section includes:

1. Conclusion
2. Future scope

Appendix:

The appendix section of a document typically contains supplementary materials that support or enhance the main content. These materials may include additional data, charts, graphs, tables, code samples, or other relevant information that provides further context or detail. The appendix serves as a reference for readers who may want to delve deeper into specific aspects of the document or need additional information to better understand the content.

References:

The references section of a document lists all the sources cited or consulted during the research or writing process. It provides readers with the necessary information to locate and verify the credibility of the sources used in the document. References are typically listed alphabetically by the author's last name or by the title of the source if no author is available. Each reference includes essential details such as the author's name, publication year, title of the work, publisher, and relevant page numbers or URLs.

CHAPTER 2

SOFTWARE REQUIREMENT SPECIFICATION

A Software Requirement Specification (SRS) defines the functional and non-functional requirements of a system. This chapter outlines the functions, users, inputs, outputs, software and hardware of PharmaCompare.

2.1 FUNCTIONAL OVERVIEW

PharmaCompare takes a prescription image and returns an analysed, checked and priced list of medicines. Its key functionalities include:

1. **User Registration and Login:**
   - Accounts are created with a name, e-mail address and password, and login returns a JSON Web Token that protects every other request.
2. **Prescription Upload:**
   - A PNG, JPEG or PDF file of up to 10 MB is stored privately under the user's own folder.
3. **Text Extraction:**
   - The text is read by Google Cloud Vision, or by EasyOCR when the cloud service is not available, and is grouped into lines with a confidence for each line.
4. **Medicine Detection:**
   - Each line is matched to the reference table of medicines and receives a match status (*matched*, *user confirmed*, *needs verification* or *not identified*); the dose, schedule and duration written beside it are read as well.
5. **Verification:**
   - The user confirms an uncertain line from the candidates shown, and a choice that differs from the line needs an explicit confirmation.
   - Every decision is recorded in an audit trail.
6. **Safety Screening:**
   - Confirmed medicines are checked for documented interactions, duplicate ingredients and declared allergies.

7. **Price Comparison and Alternatives:**
    - Each confirmed medicine is compared with products of the same ingredients, strength and form, and up to three cheaper comparable options are shown with the source of every price.
8. **Smart Health Report:**
    - The analysis is shown on screen and can be downloaded as a PDF with the summary, the medicine timeline, the safety findings and the savings.
9. **Pharmacy Stock Reports:**
    - A pharmacy can report prices and stock through a keyed feed; no feed is connected, so availability is not tracked.

2.2 USER CHARACTERISTICS

The system is designed for people who are not technical and who may be anxious about a medical document. The primary users include:

1. **Patient or Caregiver (Primary User):**
   Holds a prescription and wants to understand it and to compare prices. No medical or technical knowledge is assumed.
2. **Pharmacist or Other Health Professional (Secondary User):**
   May use the report as a starting point for a conversation about alternatives; the report states clearly what it has and has not verified.
3. **Administrator / Developer:**
   Maintains the reference datasets, the database and the OCR configuration.
4. **Basic Digital Literacy:**
   All users should be able to use a web browser, take or select a photograph, and read a table.

2.3 INPUT REQUIREMENTS

Inputs to the system are of two kinds: those supplied by the user and the reference data held by the system.

2.3.1 USER INPUT REQUIREMENTS

Table 2.1 User input requirements

Input	Details and limits	Used for
Registration	Name, e-mail (unique), password of at least 8 characters; phone is optional	Account creation
Login	E-mail and password	Authentication
Prescription file	PNG, JPG, JPEG or PDF, up to 10 MB; optional name of up to 255 characters; optional prescription type and symptoms (stored as supplied)	OCR and analysis
Allergy declaration	Names or groups of drugs, edited on the profile page	Allergy check
Verification choice	A candidate or a searched product, then Confirm; or "none of these"	Settling a line
Medicine added by hand	A medicine chosen from the search results	Adding a missed line
Details typed by the user	Name, optional strength and form, schedule and duration of a medicine the dataset lacks	Lines the dataset cannot identify

2.3.2 REFERENCE DATA REQUIREMENTS

Table 2.2 Reference data and service requirements

Source	Content	Purpose
Medicine dataset	253,973 Indian medicine products (community compiled, MIT licence): name, composition, ingredients, form, strength, manufacturer, pack size, price, discontinued flag	Matching, prices, alternatives, search
DDInter 2.0	160,235 unique interaction pairs over 1,939 names, with severity (Major, Moderate, Minor, Unknown)	Interaction screening
Google Cloud Vision (optional)	Document text detection in English and Hindi; needs an API key and sends the image to Google	Primary OCR engine
EasyOCR	English text detection and recognition; runs locally	Fallback OCR engine
Pharmacy feeds (optional)	Prices and stock that a pharmacy reports, by CSV file or keyed request; none is connected	Pharmacy-reported prices and stock
Google Gemini (optional)	Language model that words the medicine explanations; a built-in explainer is used when no key is set	Explain this medicine

2.4 OUTPUT REQUIREMENTS

The outputs include the analysed medicines, the safety findings, the comparable prices and the Smart Health Report.

2.4.1 SCREEN AND REPORT OUTPUT

Table 2.3 Screen and report output

Output	Description
Detected medicines table	Each line read from the prescription with its strength, form, match status, confidence and the reasons, reference price, cheapest comparable price and saving
Status legend	A short explanation of every status word shown in the table
Summary panel	Estimated savings, number of medicines detected, matched automatically and needing verification
Safety and interactions	One entry per finding with severity, explanation, recommendation and data source
Alternatives	Up to three cheaper comparable products per medicine, with the reason for each
Medicine timeline	The identified medicines by time of day, with the amount and duration written on the prescription; lines not identified are listed apart
Price sources notice	Where each price came from, when it was imported and whether any live feed exists
Original prescription text	The OCR lines, with uncertain lines marked
Smart Health Report (PDF)	Summary, medicines with status and dosing, timeline, safety findings and savings, with a page number on each page and the reminder that price information is indicative only

2.4.2 SAFETY ALERTS AND NOTICES

The system gives clear and concise safety messages:

- A documented interaction is shown with the severity given by DDInter (Major, Moderate, Minor or Unknown).
- A pair with no record is shown as “needs verification”, never as safe.
- Every finding advises consulting a doctor or pharmacist and never tells the user to stop, start or change a medicine.
- Every price is labelled as coming from a community dataset, not from live pharmacy stock.

2.5 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional requirements outline the specific functionalities that a software system must possess to fulfill its intended purpose and meet the needs of its users. Non-functional requirements describe the qualities of the system, such as security, performance and reliability.

2.5.1 FUNCTIONAL REQUIREMENTS

- **User Authentication:** Users register and log in securely; a prescription can be viewed or deleted only by its owner.
- **Prescription Upload:** A PNG, JPEG or PDF file is accepted; a page that is too unclear to read is rejected and a clearer image is requested.
- **OCR Processing:** Text is extracted by Google Cloud Vision or EasyOCR and grouped into lines with a confidence.
- **Medicine Identification:** Each line is matched to the medicine table, with its strength, dose, schedule and duration, and receives a match status.
- **Confidence-Based Verification:** Only matched or user-confirmed lines reach the later stages; a choice that differs from the line needs explicit confirmation and is recorded.
- **Safety Check:** Confirmed medicines are checked for documented interactions, duplicate ingredients and declared allergies.
- **Price Comparison and Recommendation:** Cheaper products of the same composition, strength and form are listed with the source of each price.
- **Smart Health Report:** The screen and the PDF show the same status wording and savings, and no price is presented as live or any medicine as available.
- **Database Management:** Users, prescriptions, results and reports are stored in MySQL.

2.5.2 NON-FUNCTIONAL REQUIREMENTS

- **User-Friendly Interface:** Single-page React application with a guided upload journey, plain-language labels and a light and a dark theme.
- **Security:** Salted password hashes, JWT tokens, owner checks, upload limits and hashed partner keys; API keys never appear in logs.
- **Performance:** Indexed medicine search; the first OCR request after start-up is slower because the OCR libraries load.

- **Accuracy:** Cautious matching with user confirmation and strict price-comparability rules; accuracy on real prescriptions is not yet measured.
- **Reliability:** One failing stage does not discard earlier results, and OCR falls back to a local engine.
- **Scalability:** A stateless Flask API and an indexed relational database.
- **Maintainability:** Separate service modules, schema and migration scripts, and automated tests.

2.6 SOFTWARE REQUIREMENTS

Table 2.4 Software requirements

Software	Requirement
Operating system	Windows 10 or above
Programming languages	Python and JavaScript (JSX)
Back-end framework	Flask (REST API) with JWT authentication
Front end	React.js, React Router and Axios, built with Vite (Node.js)
Database	MySQL
OCR	Google Cloud Vision (optional, English and Hindi) and EasyOCR
Image and PDF handling	Pillow, OpenCV and pypdfium2
Report generation	ReportLab
Medicine explanation	Google Gemini API (optional)
Web browser	Chrome, Firefox or Edge

2.7 HARDWARE REQUIREMENTS

Table 2.5 Hardware requirements

Hardware	Requirement
Processor	AMD Ryzen 5 5500U (six cores) or higher
RAM	16 GB
Storage	Free disk space for the database, uploaded files and OCR model
Network	Internet connection for the optional cloud services
Client device	A web browser, and a camera or scanner for the prescription

2.8 PROJECT CYCLE

The project cycle, also known as the software development life cycle (SDLC), refers to the stages involved in the development of a software project from initiation to completion. PharmaCompare followed a **phased development cycle**, with a feedback loop from testing back to implementation. Dates up to August 2026 follow the project schedule.

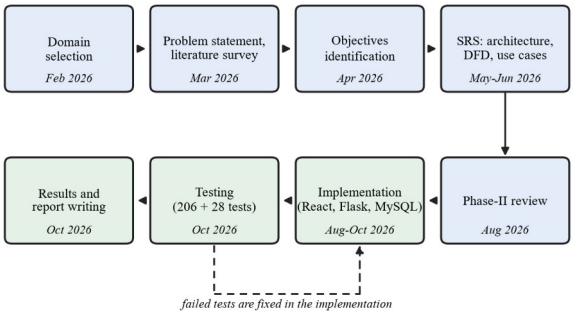


Figure 2.1 Project cycle

Figure 2.1 represents the project cycle, which comprises the following phases:

- **Domain selection (Feb 2026):** healthcare informatics and prescription intelligence.
- **Problem statement and literature survey (Mar 2026):** survey of 25 research papers.
- **Objectives identification (Apr 2026):** the objectives of Section 1.3.
- **SRS, architecture, DFD and use cases (May–Jun 2026):** requirements and design.
- **Phase-II review (Aug 2026):** review of the requirements and design.
- **Implementation (Aug–Oct 2026):** React front end, Flask API and MySQL database.
- **Testing (Oct 2026):** automated tests; failures fixed in the implementation.
- **Results and report (Oct 2026):** two dataset-level experiments and the report.

CHAPTER 3

SYSTEM DESIGN

System design is the blueprint of the system: its architecture, the way data are stored and how data and control flow through the components. This chapter covers the project architecture, the database design, the sequence diagram, the data flow diagrams, the database table structure, the flow chart and the utilities of PharmaCompare.

3.1 PROJECT ARCHITECTURE

PharmaCompare is a **three-tier web application**: a React single-page application (presentation), a Flask REST API (application) and a MySQL database (data) that also holds the two static reference datasets. The services behind the API correspond to the stages of the workflow.

Working principle:

- **Presentation layer:** the React application handles registration, upload, verification, reports, medicine search, medicine scan, the profile and the data-sources page, and talks to the API through JSON requests that carry a token.
- **Application layer:** the Flask API authenticates each request with a **JSON Web Token**, validates input and calls the services; every prescription is checked against its owner.
- **OCR service:** reads the file with **Google Cloud Vision (English and Hindi) or EasyOCR**, groups fragments into lines, decides whether the page is readable and adds photo-quality notes.
- **Medicine matching:** retrieves and scores candidates from the *Medicines* table and gives each line a **match status**.
- **Dosing reading and timeline:** reads the dose, schedule and duration written beside each medicine and builds the medicine timeline.
- **Clinical decision engine:** screens confirmed medicines for interactions, duplicate ingredients, strength inconsistencies and allergies.
- **Price comparison and alternatives:** compares each confirmed medicine with products of the **same composition**, states where each price came from, flags implausible prices and lists up to three cheaper options.
- **Verification and audit:** applies the user's decisions, warns about choices that differ from the line, accepts typed details and records every decision in *System_Logs*.

- **Scan, search and explain:** reads a pack photo once and offers candidates; searches and explains medicines.
- **Smart Health Report:** renders the results into a PDF with ReportLab.
- **Data layer:** a MySQL database of sixteen tables, including the reference tables *Medicines* (253,973 products) and *Drug_Interactions* (160,235 pairs) and the tables for pharmacy reports and data sources.
- **External services:** Google Cloud Vision (primary OCR), EasyOCR (local fallback) and Gemini (optional, summary and medicine explanations only).

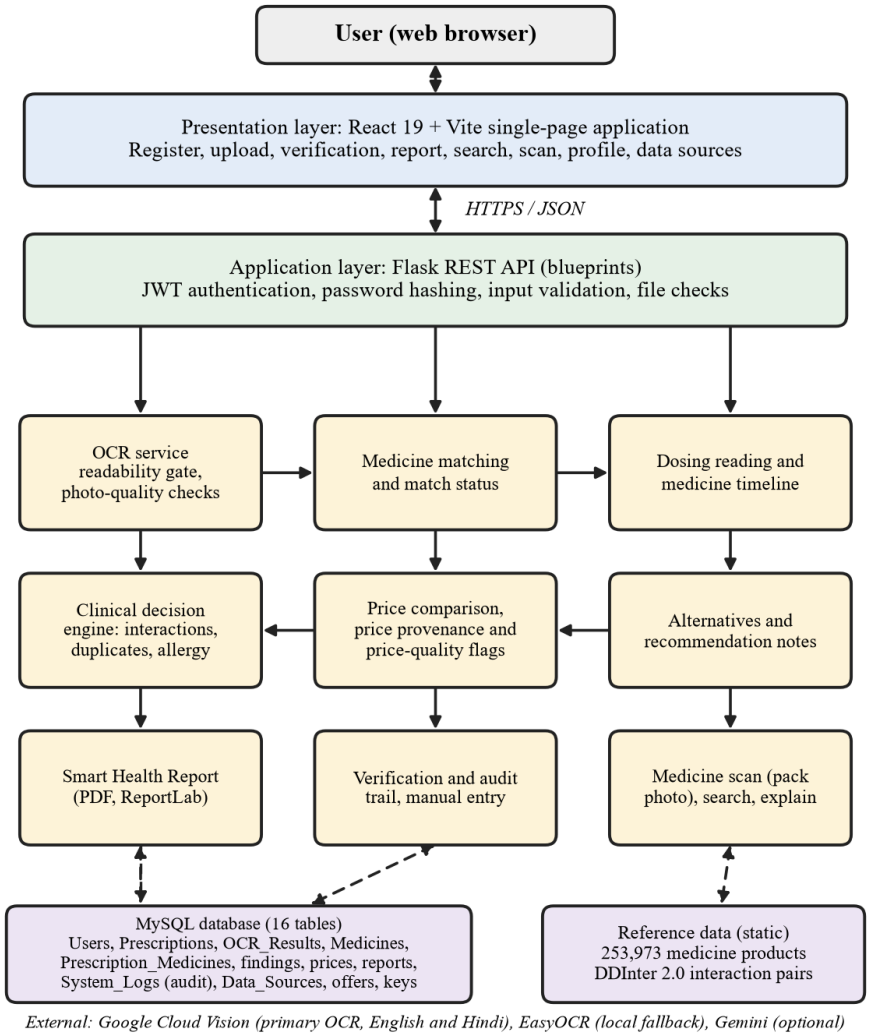


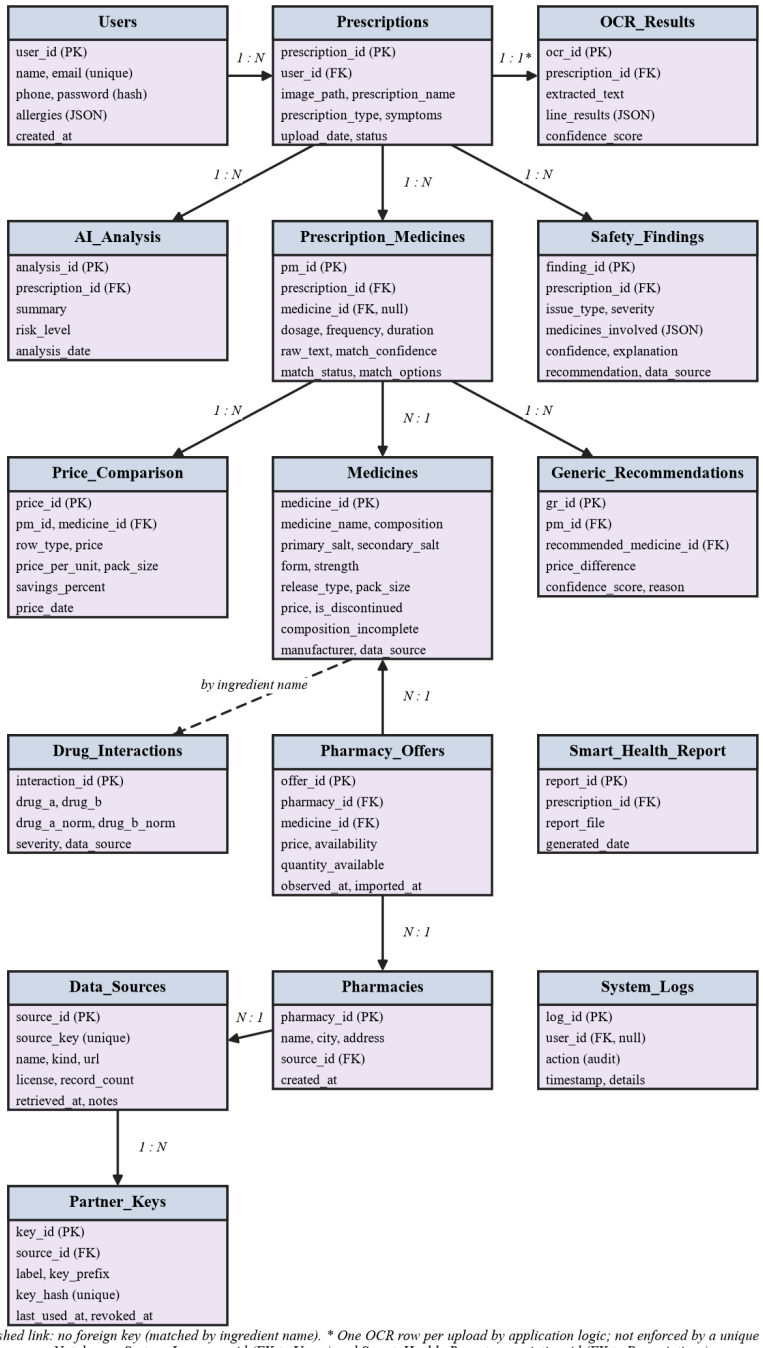
Figure 3.1 Project architecture

3.2 DATABASE DESIGN

The Phase-2 design had ten tables. **Drug_Interactions** and **Safety_Findings** were added for the clinical decision engine, and **Data_Sources**, **Pharmacies**, **Pharmacy_Offers** and **Partner_Keys** for data sources and pharmacy reports, giving sixteen. Table 3.1 gives the purpose of each table and Figure 3.2 shows their relationships.

Table 3.1 Database tables and their purpose

Table	Purpose
Users	Registered accounts, with their declared allergies
Prescriptions	One row per upload: file path, label and status
OCR_Results	Extracted text, per-line confidences and page confidence
Medicines	Reference table of 253,973 medicine products with prices
Prescription_Medicines	Detected lines: raw text, match status and chosen product
Drug_Interactions	DDInter 2.0 interaction pairs with their severity
AI_Analysis	Summary and overall risk level of a prescription
Safety_Findings	One row per safety finding, with severity and advice
Price_Comparison	Prescribed and alternative prices, unit price and saving
Generic_Recommendations	Up to three cheaper comparable options per line
Smart_Health_Report	Path and date of the latest PDF report
System_Logs	Audit trail of verification decisions: the line as read, its status and the warnings shown
Data_Sources	Registry of the datasets and feeds used, with licence, record count and when obtained
Pharmacies	Pharmacies that have reported prices or stock, with the feed that reported them
Pharmacy_Offers	Prices and stock a pharmacy reported, with the time observed; empty while no feed is connected
Partner_Keys	Hashed keys that allow a pharmacy system to send reports



*Dashed link: no foreign key (matched by ingredient name). * One OCR row per upload by application logic; not enforced by a unique key. Not drawn: System_Logs.user_id (FK to Users) and Smart_Health_Report.prescription_id (FK to Prescriptions).*

Figure 3.2 Entity-relationship diagram

3.3 SEQUENCE DIAGRAM

A sequence diagram shows the order in which the components exchange messages. Figure 3.3 follows one use of the system; the numbers refer to the messages in the diagram.

1. **Login (1–4):** the API checks the password hash in MySQL and returns a **JSON Web Token**.
2. **Upload (5–7):** the API stores the file and inserts the *Prescriptions* row.
3. **OCR (8–11):** the image goes to the OCR engine; the returned words are grouped into lines, checked for readability and stored.
4. **Matching and analysis (12–15):** each line is matched; safety, price, alternatives and the timeline are built for the settled lines, stored and returned.
5. **Verification (16–22):** the user chooses a product for an unsure line; if it conflicts with the line the API returns the differences, the user acknowledges them, the decision is saved and logged, and the analysis is re-run.
6. **Report (23–25):** the user downloads the **Smart Health Report**.

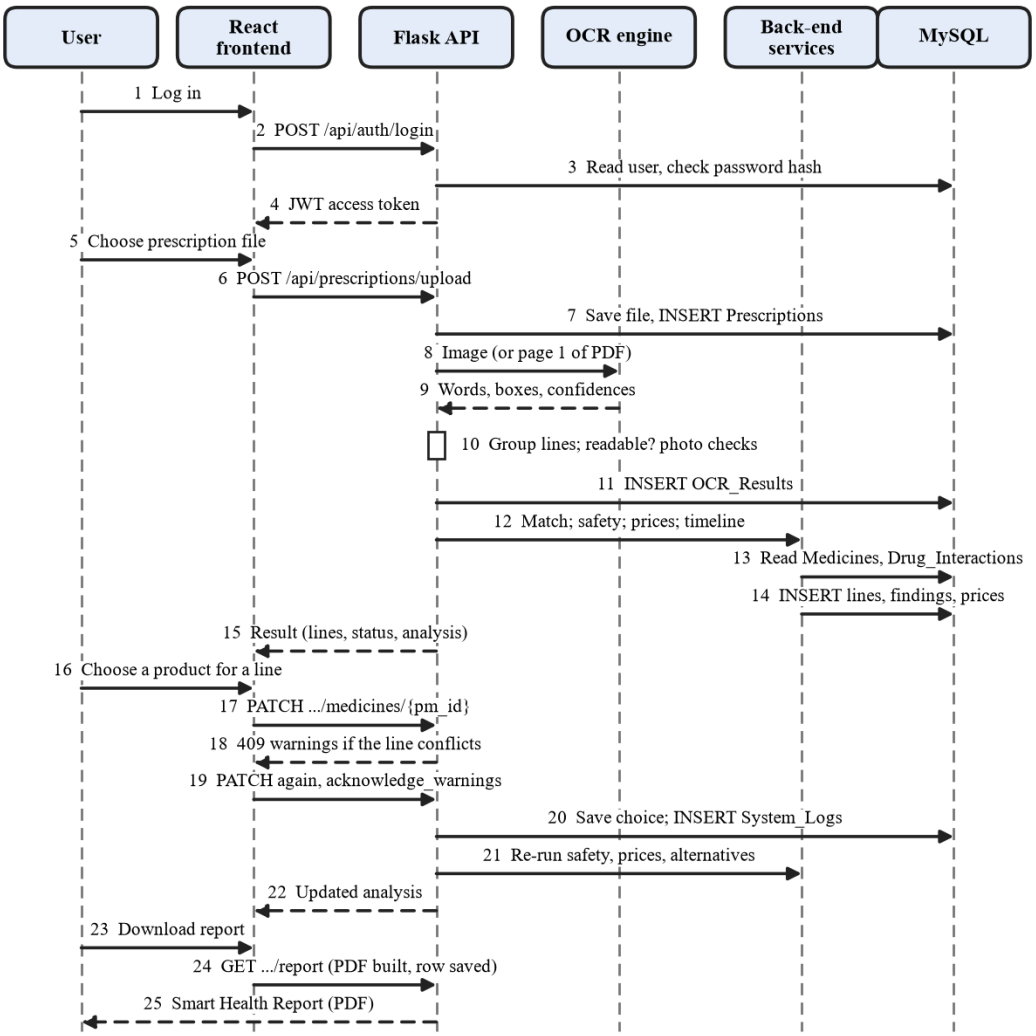


Figure 3.3 Sequence diagram

3.4 DATA FLOW DIAGRAM

A data flow diagram shows how data move between the user, the processes and the data stores. The context diagram (level 0) treats the whole system as one process; the level-1 diagram opens it into its main processes.

Level 0 - context diagram (Block Diagram)

The input, process and output of the system are:

- **Input:** the prescription file, the declared allergies and the user's confirmations.
- **Process:** OCR, medicine matching, safety screening and price comparison.
- **Output:** the analysed medicines, safety findings, prices and alternatives, and the **Smart Health Report**.

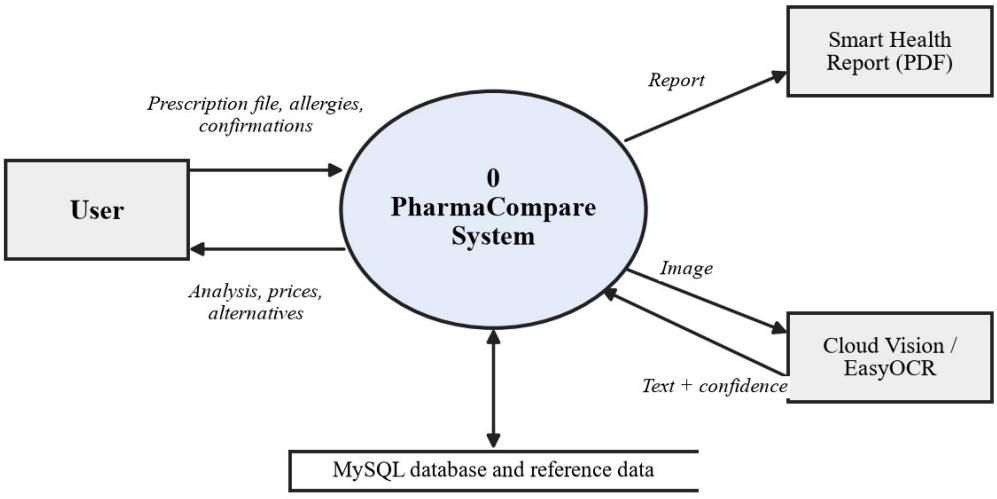


Figure 3.4 Data flow diagram, level 0

Level 1 - main processes

The level-1 diagram opens the system into five processes:

- **1.0 Capture and extract (OCR):** reads the file and stores the text lines and confidences (D1).
- **2.0 Detect and confirm medicines:** matches lines to the reference data (D2); stores their status and the audit log (D3) and takes the user's confirmations.
- **3.0 Analyze safety:** checks the confirmed lines against the interaction pairs (D4); findings go to D5.
- **4.0 Compare prices and recommend:** compares prices and alternatives and stores them (D6); it also reads pharmacy reports and data sources (D7).
- **5.0 Generate report:** collects the results and records the PDF (D8).

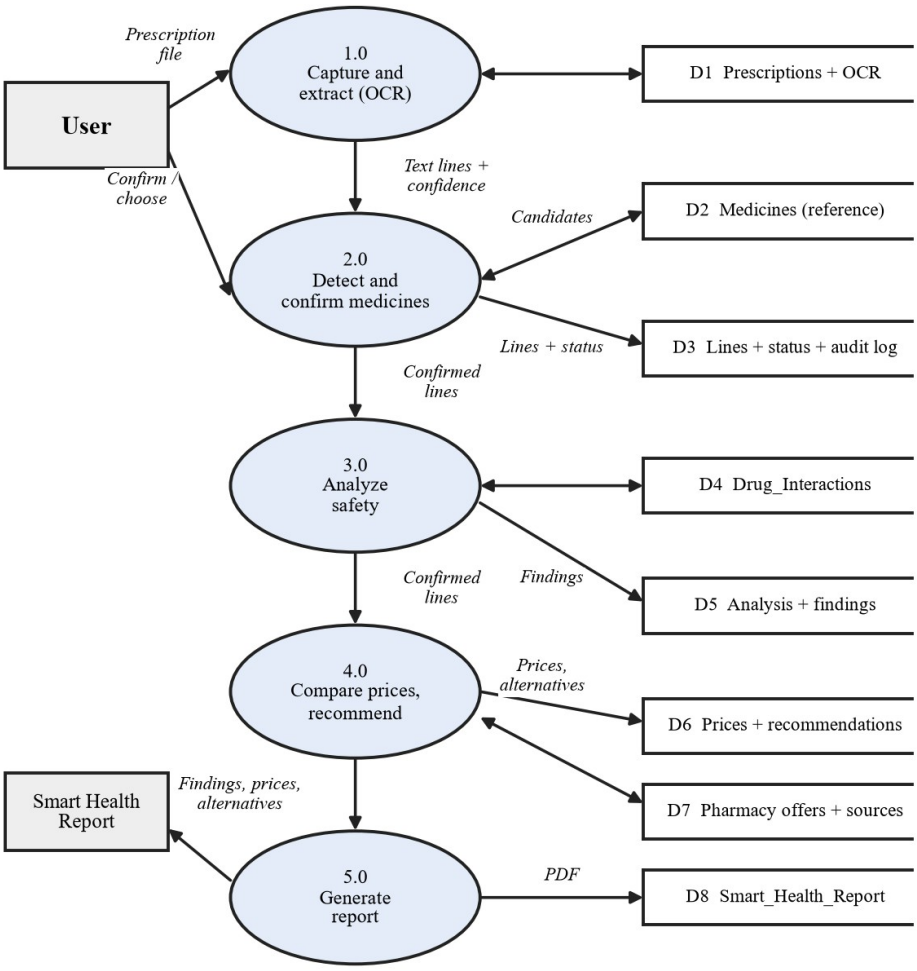


Figure 3.5 Data flow diagram, level 1

3.5 DATABASE TABLE STRUCTURE

Table 3.2 gives the structure of the database tables, taken from the live MySQL schema. All tables use InnoDB and utf8mb4, and foreign keys use *ON DELETE CASCADE* (*ON DELETE SET NULL* for *System_Logs.user_id*), so **deleting a prescription removes everything derived from it**. The tables for pharmacy reports start empty.

Table 3.2 Structure of the database tables

Users

Column	Data type	Key	Description
user_id	INT	PK	Auto-increment identifier
name	VARCHAR(150)		Full name
email	VARCHAR(150)	Unique	Login identifier, unique
phone	VARCHAR(20), NULL		Optional phone number
password	VARCHAR(255)		Salted password hash (never the password)
created_at	DATETIME		Account creation time
allergies	JSON, NULL		Self-declared allergies; empty if never declared

Prescriptions

Column	Data type	Key	Description
prescription_id	INT	PK	Auto-increment identifier
user_id	INT	FK	Owner of the prescription (Users)
image_path	VARCHAR(500)		Private path of the stored file, with a random name
prescription_name	VARCHAR(255), NULL		Label chosen by the user
prescription_type	VARCHAR(100), NULL		Optional context supplied by the user
symptoms	JSON, NULL		Optional symptoms supplied by the user
upload_date	DATETIME		Upload time
status	VARCHAR(50)		uploaded, processed, needs_reupload or processing_failed

OCR_Results

Column	Data type	Key	Description
ocr_id	INT	PK	Auto-increment identifier
prescription_id	INT	FK	Prescription that was read (Prescriptions)
extracted_text	TEXT, NULL		Text of the whole page
line_results	JSON, NULL		The same text as lines, each with its own confidence
confidence_score	DECIMAL(5,2), NULL		Mean line confidence of the page
processed_date	DATETIME		Time of processing

Medicines

Column	Data type	Key	Description
medicine_id	INT	PK	Auto-increment identifier
medicine_name	VARCHAR(255)		Brand or product name (indexed)
composition	VARCHAR(300), NULL		Ingredients with strengths, e.g. Paracetamol (650mg)
form	VARCHAR(50), NULL		Dosage form, derived from the name
strength	VARCHAR(50), NULL		First strength
category	VARCHAR(100), NULL		Category (not used by the application)
primary_salt	VARCHAR(150), NULL		First ingredient (indexed)
secondary_salt	VARCHAR(150), NULL		Second ingredient
manufacturer	VARCHAR(150), NULL		Maker
pack_size	VARCHAR(100), NULL		Pack, e.g. strip of 15 tablets
release_type	VARCHAR(12), NULL		sr, er, cr, pr, dt, odt, chewable, ec, mr or modified; empty for conventional release
composition_incomplete	BOOLEAN		Set when the name shows three or more strengths but the composition lists at most two ingredients
price	DECIMAL(10,2), NULL		Single reference price of the pack
is_discontinued	BOOLEAN		Discontinued flag from the dataset
source_dataset_id	VARCHAR(20), NULL		Identifier in the source dataset
data_source	VARCHAR(150)		Source tag of the row
imported_at	DATETIME, NULL		Import time

Prescription_Medicines

Column	Data type	Key	Description
pm_id	INT	PK	Auto-increment identifier
prescription_id	INT	FK	Prescription the line belongs to (Prescriptions)
medicine_id	INT, NULL	FK	Matched or confirmed product; empty while unconfirmed (Medicines)
dosage	VARCHAR(100), NULL		Dose read from the line; empty if not stated
frequency	VARCHAR(100), NULL		Frequency read from the line; empty if not stated
duration	VARCHAR(100), NULL		Duration read from the line; empty if not stated
raw_text	VARCHAR(500), NULL		The line as read by OCR

Column	Data type	Key	Description
match_confidence	DECIMAL(5,4), NULL		Score of the match
match_status	VARCHAR(30)		matched, user_confirmed, needs_verification or user_unmatched
match_options	JSON, NULL		Look-alike candidates with reasons, runner-up products, or details typed by the user

Drug_Interactions

Column	Data type	Key	Description
interaction_id	INT	PK	Auto-increment identifier
drug_a	VARCHAR(150)		First substance, as named by DDInter
drug_b	VARCHAR(150)		Second substance, as named by DDInter
drug_a_norm	VARCHAR(150)		Lower-case form of drug_a used for matching (indexed)
drug_b_norm	VARCHAR(150)		Lower-case form of drug_b used for matching (indexed)
severity	VARCHAR(30)		DDInter's own scale: Major, Moderate, Minor or Unknown
data_source	VARCHAR(150)		Source tag of the row
imported_at	DATETIME, NULL		Import time

AI_Analysis

Column	Data type	Key	Description
analysis_id	INT	PK	Auto-increment identifier
prescription_id	INT	FK	Prescription analysed (Prescriptions)
summary	TEXT, NULL		Summary of the safety analysis
risk_level	VARCHAR(50), NULL		Overall risk level (that of the most severe finding)
analysis_date	DATETIME		Time of the analysis

Safety_Findings

Column	Data type	Key	Description
finding_id	INT	PK	Auto-increment identifier
prescription_id	INT	FK	Prescription analysed (Prescriptions)
issue_type	VARCHAR(30)		interaction, duplicate, dosage_inconsistency or allergy_check
medicines_involved	JSON, NULL		The medicines concerned (id and name)

Column	Data type	Key	Description
severity	VARCHAR(30)		Major, Moderate, Minor, Unknown or NEEDS_VERIFICATION
confidence	DECIMAL(5,4), NULL		Confidence of the finding
explanation	TEXT		What was found
recommendation	TEXT		Advice, always to consult a doctor or pharmacist
data_source	VARCHAR(150)		Where the finding came from
created_at	DATETIME		Time the finding was created

Price_Comparison

Column	Data type	Key	Description
price_id	INT	PK	Auto-increment identifier
pm_id	INT	FK	Prescription line the row belongs to (Prescription_Medicines)
medicine_id	INT	FK	Product the row describes (Medicines)
row_type	VARCHAR(20)		prescribed or alternative
pharmacy_name	VARCHAR(150), NULL		Left empty: only one reference price per product exists
price	DECIMAL(10,2)		Pack price
price_per_unit	DECIMAL(10,4), NULL		Price of one unit of the pack
pack_size	VARCHAR(100), NULL		Pack of the product
price_difference	DECIMAL(10,2), NULL		Difference to the prescribed product; empty on the prescribed row
savings_percent	DECIMAL(6,2), NULL		Saving over the prescribed product
is_discontinued	BOOLEAN		Discontinued flag of the product
data_source	VARCHAR(150)		Source tag of the price
price_date	DATETIME		When the price was captured in the source dataset, not today
last_updated	DATETIME		Time the row was written

Generic_Recommendations

Column	Data type	Key	Description
gr_id	INT	PK	Auto-increment identifier
pm_id	INT	FK	Prescription line (Prescription_Medicines)
recommended_medicine_id	INT	FK	Comparable product suggested (Medicines)
generic_name	VARCHAR(200)		Name of the suggested product
price_difference	DECIMAL(10,2), NULL		Saving in rupees
confidence_score	DECIMAL(5,2), NULL		Bookkeeping score, not a probability
reason	TEXT		The facts behind the suggestion
data_source	VARCHAR(150)		Source tag
created_at	DATETIME		Time the row was written

Smart_Health_Report

Column	Data type	Key	Description
report_id	INT	PK	Auto-increment identifier
prescription_id	INT	FK	Prescription the report belongs to (Prescriptions)
report_file	VARCHAR(500), NULL		Path of the generated PDF
generated_date	DATETIME		Time of generation

System_Logs

Column	Data type	Key	Description
log_id	INT	PK	Auto-increment identifier
user_id	INT, NULL	FK	User concerned; set to empty if the user is deleted (Users)
action	VARCHAR(150)		Action name, e.g. medicine_verification
timestamp	DATETIME		Time of the action
details	TEXT, NULL		Details; JSON for a verification decision

Data_Sources

Column	Data type	Key	Description
source_id	INT	PK	Auto-increment identifier
source_key	VARCHAR(80)	Unique	Unique key of the source
name	VARCHAR(200)		Name of the source
kind	VARCHAR(30)		reference_dataset, interaction_dataset, pharmacy_feed or synthetic_demo

Column	Data type	Key	Description
url	VARCHAR(500), NULL		Where the data came from
license	VARCHAR(200), NULL		Licence as recorded; empty when not recorded
record_count	INT, NULL		Number of records
retrieved_at	DATETIME, NULL		When the data were obtained, not when they were true
notes	TEXT, NULL		Measured quality notes
created_at	DATETIME		Time the row was written

Pharmacies

Column	Data type	Key	Description
pharmacy_id	INT	PK	Auto-increment identifier
name	VARCHAR(150)		Pharmacy name
city	VARCHAR(100)		City
address	VARCHAR(255), NULL		Address, if given
source_id	INT	FK	Feed that reported the pharmacy (Data_Sources)
created_at	DATETIME		Time the row was written

Pharmacy_Offers

Column	Data type	Key	Description
offer_id	INT	PK	Auto-increment identifier
pharmacy_id	INT	FK	Pharmacy that reported (Pharmacies)
medicine_id	INT	FK	Product reported (Medicines)
price	DECIMAL(10,2), NULL		Reported price; empty when only stock was reported
availability	VARCHAR(20)		in_stock, limited, out_of_stock or unknown
quantity_available	INT, NULL		Quantity, if reported
observed_at	DATETIME		When the pharmacy observed it (required)
imported_at	DATETIME		Time the report was received

Partner_Keys

Column	Data type	Key	Description
key_id	INT	PK	Auto-increment identifier
source_id	INT	FK	Feed the key belongs to (Data_Sources)
label	VARCHAR(100)		Name of the key
key_prefix	CHAR(8)		First characters of the key, to identify it
key_hash	CHAR(64)	Unique	Hash of the key; the key is never stored
created_at	DATETIME		Time the key was created
last_used_at	DATETIME, NULL		Time of the last request
revoked_at	DATETIME, NULL		Time the key was revoked

3.6 FLOW CHART

The flow chart in Figure 3.6 represents the logical steps of the main user flow. The process starts with the user logging in, because every upload, verification and report request needs a valid token.

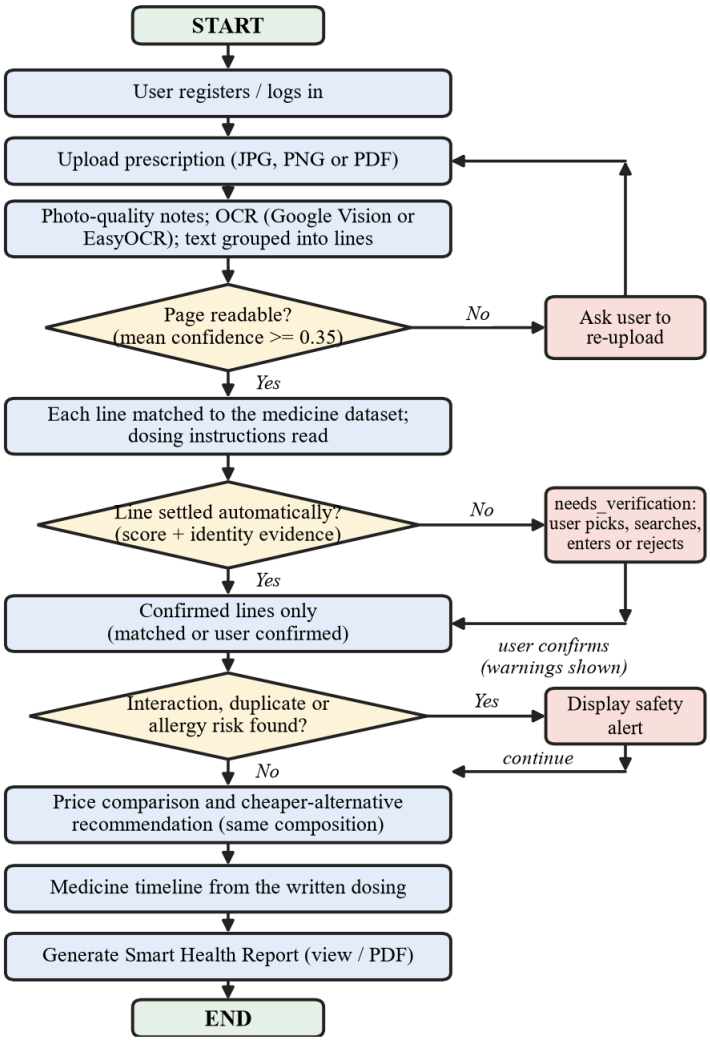


Figure 3.6 Flow chart

Description of the flow:

- **Start:** the user registers or logs in and uploads a JPG, PNG or PDF file.
- **OCR and quality check:** photo-quality notes are added and the lines are read; a page with a mean confidence below 0.35 is rejected and a clearer image is requested.
- **Medicine detection:** lines are matched to the medicine dataset and their dosing is read; a line that is not settled becomes *needs verification* and waits for the user, who may choose, type in or reject a product.

- **Confirmed lines only:** only *matched* or *user confirmed* lines continue; a choice that differs from the line is confirmed only after the warnings are acknowledged.
- **Safety check:** an interaction, duplicate or allergy risk shows a safety alert and the analysis continues.
- **Price, timeline and report:** same-composition prices and up to three cheaper options are listed, the timeline is built and the Smart Health Report is generated. **End.**

3.7 UTILITIES

Besides the main stages, the back end contains small utility modules that several stages depend on. They are listed in Table 3.3.

Table 3.3 Utility modules of the back end

Module	Function
dosage_parser	Reads the frequency and the duration from the text of a line; nothing is inferred
instruction_service	Attaches dosing rows to the medicine above them and reads schedules; a doubtful reading is marked as possible
timeline_service	Builds the medicine timeline once, for both the screen and the PDF
prescription_notes_service	Lists lines that are not medicines (instructions, Hindi notes) apart from the medicines
image_quality_service	Checks sharpness, size, brightness and contrast of a photo and says what to fix; it only warns
release_type	Derives the release mechanism (SR, ER, CR and so on) of a product and detects a release marker on a line
composition	Splits a composition into ingredients and strengths and flags incomplete ingredient lists
match_status	The one vocabulary for how a line came to point at a medicine
manual_entry_service	Stores the details a user types for a medicine the dataset lacks; never matched or priced
medicine_search	Deterministic ranking for the “which medicine is this?” picker; no fuzzy guesses
scan_service	Reads a pack photo, offers candidates through the matcher and deletes the photo
price_quality	Parses pack sizes, computes the price per unit and flags extreme prices
price_provenance_service	States the source, import date and “not live” status of every price
recommendation_notes	The wording of what an alternative does and does not establish, shared by screen and PDF
offer_service	Words what pharmacies have reported, with its age; no report means “not reported”
feed_import_service / partner_feed_service	Validate pharmacy feeds strictly; a row is accepted only if every part is certain
data_validation_service	Checks the medicine data against its own fields and notes what to check
data_sources_service	Lists the data sources with licence, record count and freshness
allergy_service	Compares declared allergies with ingredients, including the penicillin, sulfonamide and NSAID groups
medicine_explainer / report_summary_service	Plain-language explanations and summary; Gemini when a key is set, otherwise written from the stored facts
display_rounding	One way of rounding a percentage, so that the screen and the PDF agree
smart_report_service	Builds the PDF report from results that were already computed
cloud_ocr_service	Calls Google Cloud Vision and removes the API key from any message, log or error

CHAPTER 4

IMPLEMENTATION

This chapter discusses the software tools used, the implementation of each module with the rules and thresholds that it applies, and the design of the front-end screens.

4.1 SOFTWARE TOOLS USED

Table 4.1 Software tools used

Tool / library	Purpose / description
Python	Language of the back end, the services, the import scripts and the tests
Flask and Flask-CORS	REST API with blueprints for authentication, prescriptions, analysis, prices and recommendations
Flask-JWT-Extended	Issues and verifies JSON Web Tokens; Werkzeug supplies the salted password hashing
MySQL and mysql-connector-python	Relational database, full-text index on medicine names and compositions, and the data access layer
EasyOCR (PyTorch)	Local OCR engine; the model is loaded once and kept in memory
Google Cloud Vision (REST)	Document text detection with English and Hindi language hints; used first when a key is configured
pypdfium2, Pillow, OpenCV	Renders the first page of a PDF and prepares images for OCR
difflib (Python standard library)	Similarity ratio used to score candidate medicines
ReportLab	Generates the Smart Health Report PDF
Google Gemini API (optional)	Words the “explain this medicine” text and the report summary from stored facts only
React, React Router, Axios	Single-page front end, routing and API calls
Vite and oxlint	Front-end build tool, development server and linter
unittest and Node test runner	Back-end test suite (206 tests) and front-end logic tests (28 tests)
Git	Version control of the source code

4.2 IMPLEMENTATION DETAILS

The implementation follows the stages of the workflow. Each stage is a separate service module, so it can be tested and replaced on its own.

4.2.1 REFERENCE DATA IMPORT

The medicine table was imported on 28 August 2026 from a **community-compiled, MIT-licensed dataset of 253,973 products** whose original source is not stated; every row carries a source tag. The dosage form is derived from the product name, and the release type and an incomplete-composition flag were added by a migration script. The interaction data come from the eight DDInter 2.0 category files: 222,383 raw rows reduce to **160,235 unique pairs** over 1,939 names. A read-only audit script reports duplicates, blank strengths and suspicious prices without changing any data. A registry table lists each dataset with its licence (as recorded), record count and the date it was obtained, and a validation script checks every product against its own fields (12,007 products carry at least one note, such as an incomplete ingredient list or a strength that could not be matched); the notes are a reason to check, never a correction.

4.2.2 AUTHENTICATION AND SECURITY

- **Registration and login:** unique e-mail, password of at least eight characters stored as a **salted hash**; login returns an access token sent with every request.
- **Ownership:** every prescription route refuses access to another account.
- **Uploads:** only PNG, JPEG and PDF files up to 10 MB, stored under the user's folder with a random name.
- **Secrets:** API keys come from the environment; tests confirm they never appear in errors, logs or responses.

4.2.3 TEXT EXTRACTION

Google Cloud Vision runs first when a key is configured; **EasyOCR** runs locally when no key is set or the cloud call fails (either can also be pinned by a setting). The cloud request asks for English and Hindi. A PDF is rendered from its first page. Both engines return word boxes with confidences, which are grouped into printed lines so that instructions to the right of a medicine name stay on the same line. For a line made of fragments x_i with confidences c_i, the line confidence is the length-weighted mean:

$$c_\ell = \Sigma\, w_i \cdot c_i \,/\, \Sigma\, w_i\,, \quad w_i = max(|x_i|,\, 1) \tag{4.1}$$

and the line is marked uncertain if any fragment has c_i below 0.55. The page counts as readable when the mean line confidence reaches 0.35:

$$C = (1/L)\, \Sigma\, c_\ell \;\geq\; 0.35 \tag{4.2}$$

This is a confidence proxy; it does not say why a page reads badly. An unreadable page is stored with its status *needs_reupload* and the user is asked for a clearer image. Separate **photo-quality checks** look at the pixels (size, sharpness, brightness and contrast, with configurable thresholds of 600 pixels, 20, 70 and 15) and say in words what to fix. They only warn: they never block an upload, never change what the OCR reads and never claim that a photo is good.

4.2.4 MEDICINE DETECTION

Letterheads, addresses and administrative text are filtered out. For the other lines, candidates are retrieved from the *Medicines* table by a **MySQL full-text search** (15 candidates) and a **brand-prefix lookup**. Each candidate is scored with the similarity ratio of Python's sequence matcher, sim = 2M/T, against the product name and its primary ingredient. A strength, form, release marker or ingredient on the line that agrees with the candidate gives corroborating signals q_j that raise the score:

$$t = max(\, sim(\ell, name),\, sim(\ell, salt)\,), \qquad s = 0.6 \cdot t + 0.4 \cdot \bar{q} \tag{4.3}$$

where $\bar{q}$ is the mean of the positive signals. The bar is **0.60 with corroboration and 0.78 on text alone**. A candidate is disqualified when the line states a different strength, form or release type or an ingredient the product lacks; an unrequested combination product or a conflicting form shorthand subtracts 0.15. A match is **settled** only if the score reaches 0.90, or every word of the line occurs in the product name, or all named ingredients are present:

$$settled \Leftrightarrow (\, s \geq 0.90\,) \;\vee\; (\, exact\ words\,) \;\vee\; (\, all\ ingredients\ named\,) \tag{4.4}$$

and the dataset must support it: a product with no strength on record cannot settle a line that states one, and a product with an incomplete ingredient list is never settled automatically. Any other line gets its closest guess (score of at least 0.5) with the status *needs_verification*, and up to three options when several different products fit equally well (within 0.04). The 0.90 bar is a **design parameter**; it has not been tuned on labelled prescriptions.

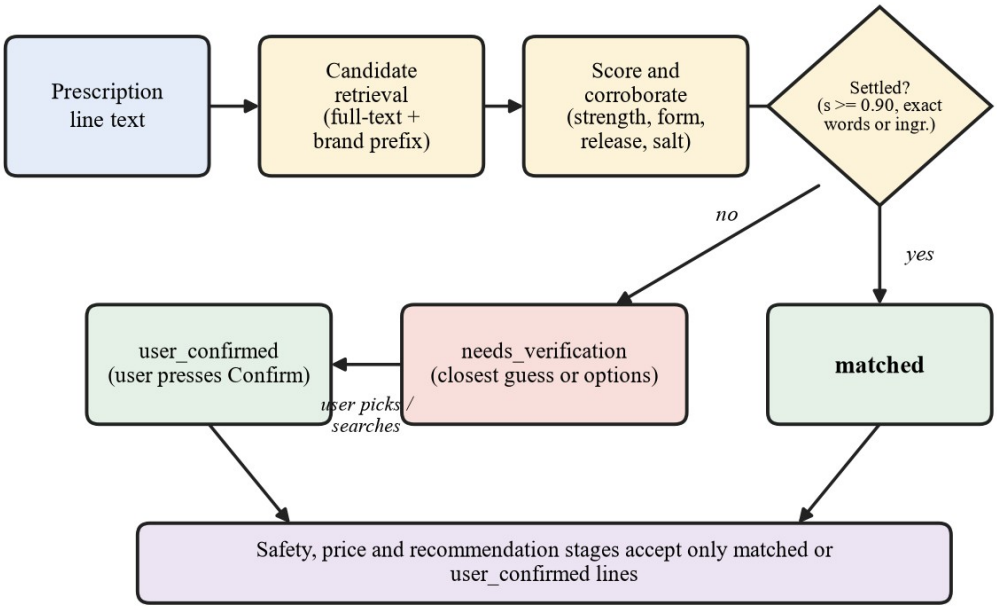


Figure 4.1 Medicine matching and match statuses

4.2.5 USER CONFIRMATION AND VERIFICATION

A line is settled automatically only when the match is *matched*. Any other line keeps the closest look-alikes with plain-language reasons and waits for the user. The user may:

- **Choose a candidate or a searched product** and press **Confirm medicine**, which sets the status *user_confirmed*;
- **Type in the details** of a medicine that the dataset lacks (name, strength, form, schedule and duration); it is shown as *entered by you* and is never matched, priced or checked for interactions;
- **Mark the line as none of these**, which sets the status *user_unmatched*.

When the chosen product differs from the line in name, form, strength, ingredient or release type, the server answers with the differences instead of saving, and the choice is saved only when the user confirms again with *acknowledge_warnings*. Such a choice is kept as the user's decision but **never counts in the headline savings**, and the screen states that confirming does not show equivalence. Every decision is written to *System_Logs* with the line as read, its status and confidence before the decision, the candidates shown and the warnings acknowledged; the original line text is never changed, and the log rows are deleted with the prescription. **Every later stage takes only matched or user_confirmed lines.** Confirming,

adding, entering or removing a line re-runs the analysis; changing the allergy profile re-runs it for all of the user's prescriptions.

4.2.6 DOSING INSTRUCTIONS, NOTES AND MEDICINE TIMELINE

The instruction is usually written to the right of, or below, the medicine name, so OCR returns it as separate lines, and handwriting is read loosely ("1-0-0" may come back as "100"). The instruction reader therefore attaches a dosing row to the medicine above it, never across another medicine, and reads only what the prescription contains:

- A schedule such as **1-0-1** is read as written; digits that ran together are only **possibly** a schedule, and a strength or a number with a unit is never read as one;
- Words such as *twice daily*, *before food* and *as needed*, a duration such as *for 5 days* and a note in Hindi that every word of which is a plain time-of-day word are read; nothing is defaulted;
- Lines that are **not medicines**, such as instructions for the nurse or notes in Hindi, are listed apart from the medicines, never matched to a product.

The **medicine timeline** is built once, for the screen and the PDF alike. Only identified medicines appear by name, placed in the morning, afternoon, evening or night from the written schedule; a line with a frequency but no schedule is listed as "time of day not written"; a line that is not identified is listed apart with what was read from it. The timeline is taken from the instructions and is never a dosing recommendation.

4.2.7 SAFETY SCREENING

For every pair of confirmed medicines with different primary ingredients, the names are looked up in the DDInter pair table, as written and then after removing salt-form words and applying a short audited alias list. One of three outcomes is returned:

- A **documented interaction**, with DDInter's severity copied unchanged (Major, Moderate, Minor or Unknown);
- **No record**, although both ingredients are tracked by DDInter;
- An **ingredient that the source does not cover**.

The last two outcomes are labelled **needs verification** and are not a statement of safety. Further checks flag **duplicate ingredients**, a **strength that differs** from the matched product's, and a **declared allergy** (by name or through the penicillin, sulfonamide and NSAID

groups). Every finding ends with advice to consult a doctor or pharmacist. About 56% of the distinct ingredients in the medicine table have a match in DDInter.

4.2.8 PRICE COMPARISON AND ALTERNATIVES

Two products i and j are **comparable** only if they share the ingredients, the full composition with strengths, the dosage form and the release type (conventional, SR, ER, CR and so on), and neither is discontinued, unpriced or recorded with an incomplete ingredient list:

$$i \approx j \Leftrightarrow comp(i) = comp(j) \wedge str(i) = str(j) \wedge form(i) = form(j) \wedge rel(i) = rel(j) \qquad (4.5)$$

Pack sizes differ, so a price P and a pack quantity n give a **unit price**, and the saving of an alternative j over the prescribed product i is

$$u = P / n , \quad saving = (P_i - P_j) / P_i \times 100 \qquad (4.6)$$

The percentage is reported only for packs of the same kind and quantity of unit, and never for a flagged price. In a comparable group of at least eight priced products, a unit price is flagged as unusually low or high when

$$u < \tilde{u} / 10 \quad or \quad u > 10 \cdot \tilde{u} \qquad (4.7)$$

where ũ is the group's **median unit price**. A flagged price is shown with a warning and never counted as a saving. Up to ten comparable alternatives are listed, and up to **three cheaper, unflagged, same-quantity options** are presented, each with a bookkeeping score (1.0 for a saving of at least 20% with at least two options, else 0.7) that is not a probability. A generic name without a brand gets no price against one arbitrary brand.

Every price states its **provenance**: a reference price is the product's price in the imported dataset, shown with the import date and its age (a snapshot older than 180 days is called old), and is never a live price. A price that a pharmacy reported is shown separately, with the pharmacy, the feed and the time observed. Next to the alternatives, notes state that an alternative has the same ingredients, strength, form and release type, what was not compared (inactive ingredients, maker quality, bioequivalence), and that the decision belongs to the prescriber or pharmacist.

4.2.9 SMART HEALTH REPORT

The **Smart Health Report** is a PDF made with ReportLab. It opens with a summary of the analysis, then lists the medicines with their status and dosing, the medicine timeline, the safety findings, the alternatives and a savings table, and says where the prices come from. Every page is numbered and carries the note that price information is indicative only. The

summary is written by **Gemini** only when a key is set, from names, strengths, schedules and counts alone; every sentence is checked against the facts, and a deterministic summary replaces it if the check fails or no key is set. The report computes nothing new, and tests check that its wording and totals equal those on screen.

4.2.10 MEDICINE SEARCH, EXPLANATION AND SCAN

Any medicine can be searched by brand, ingredient or strength: every word of the query must match, whole words rank above prefixes, and discontinued products are never first. The same search feeds the verification dialog. “Explain this medicine” describes a product in plain language, using **Gemini** (given only the stored record) when a key is set, and otherwise a **deterministic explainer** built from the same record. A **Buy Generic** button opens a web search for the product’s ingredients, strength and form in a new tab; PharmaCompare has not checked the results and shows no price or stock from them.

The **scan** reads a photo of a medicine pack once, ignores packaging text such as maker, price and batch, runs the same matcher over the remaining lines and returns the products that fit, each with its reason. It never confirms anything, cannot tell whether a pack is genuine, and the photo is deleted after it is read.

4.2.11 PHARMACY REPORTS AND DATA SOURCES

A pharmacy system can report prices and stock either as a CSV file imported by a script or by a keyed request. A row is accepted only when every part is certain: an observation time that is not in the future, one unambiguous catalogue product, and a positive price or a recognised stock word; rejected rows come back with their reasons. Keys are stored only as hashes, and a request is limited in size, rows and rate. Reports are shown with the pharmacy, the feed and their age (stock older than 48 hours and prices older than 7 days are marked old) and are never mixed with the dataset price. **No pharmacy feed is connected**, so the tables are empty, the app says that no live price or stock feed is connected, and availability is described as not tracked.

4.2.12 API ENDPOINTS AND PARAMETERS

Table 4.2 REST API endpoints

Method and path	Purpose
POST /api/auth/register, /api/auth/login	Create an account; log in and receive a token

Method and path	Purpose
GET /api/auth/me; PUT /api/auth/me/allergies	Current user; update the allergy profile
POST /api/prescriptions/upload	Upload, run OCR, detect medicines and run the analysis
GET /api/prescriptions; GET /api/prescriptions/{id}	History; one prescription with its lines and notes
GET /api/prescriptions/{id}/analysis, /prices, /recommendations	Safety analysis; price comparison; alternatives
GET /api/prescriptions/{id}/summary	Plain-language summary of the analysis
POST /api/prescriptions/{id}/medicines	Add a medicine by hand
PATCH /api/prescriptions/{id}/medicines/{pm_id}	Confirm a product for a line (differences must be acknowledged)
POST .../medicines/{pm_id}/unmatched, /details	Mark a line as none of these; save details typed by the user
DELETE /api/prescriptions/{id}/medicines/{pm_id}	Remove a line
GET /api/prescriptions/{id}/verification-log	Decision history of a prescription
GET /api/prescriptions/{id}/report	Download the Smart Health Report (PDF)
DELETE /api/prescriptions/{id}	Delete a prescription and everything derived from it
GET /api/medicines/search; POST /api/medicines/scan	Search the medicine table; read a pack photo
GET /api/medicines/{id}/explain; GET /api/medicines/{id}/offers	Explanation of a medicine; pharmacy reports for it
POST /api/feeds/offers	Receive pharmacy reports with a partner key
GET /api/data-sources	Data sources, provenance and the status of external services
GET /api/health; GET /api/db-health	Service and database status

Table 4.3 Thresholds and parameters

Parameter	Value	Used for
Page readable	mean line confidence ≥ 0.35	Rejecting unreadable pages
Uncertain line	any fragment confidence < 0.55	Marking lines for verification
Match bar, corroborated	0.60	Candidate score with strength, form or ingredient evidence
Match bar, text only	0.78	Candidate score from text similarity alone
Settled automatically	$s \geq 0.90$, or exact words, or all ingredients named	Granting status “matched”
Closest guess	$s \geq 0.50$	Showing a guess for a line that needs verification

Parameter	Value	Used for
Options margin	within 0.04, up to 3 options	Offering a choice instead of one answer
Extreme price	unit price below 1/10 or above 10 times the group median; group of at least 8	Flagging implausible prices
Recommendations	up to 3 per medicine; saving of 20% or more with at least 2 options scores 1.0	Bookkeeping score
Photo quality	at least 600 px; sharpness 20, brightness 70, contrast 15	Warnings about a poor photo
Reference price age	older than 180 days	Calling a dataset snapshot old
Pharmacy report age	stock older than 48 hours; price older than 7 days	Marking a report as old
Partner feed limits	500 rows, 300 KB, 30 requests a minute	Limiting a feed request
Upload	PNG, JPEG, PDF; 10 MB	File validation

4.3 FRONT-END FORMS DESIGN

The front end is a **React single-page application**. Public pages (landing, about, login, register) are open to everyone; the other pages sit behind a route guard. Figures 4.2 to 4.11 show the main forms, captured from the running application with a test account; the analysis screens are shown in Chapter 6.

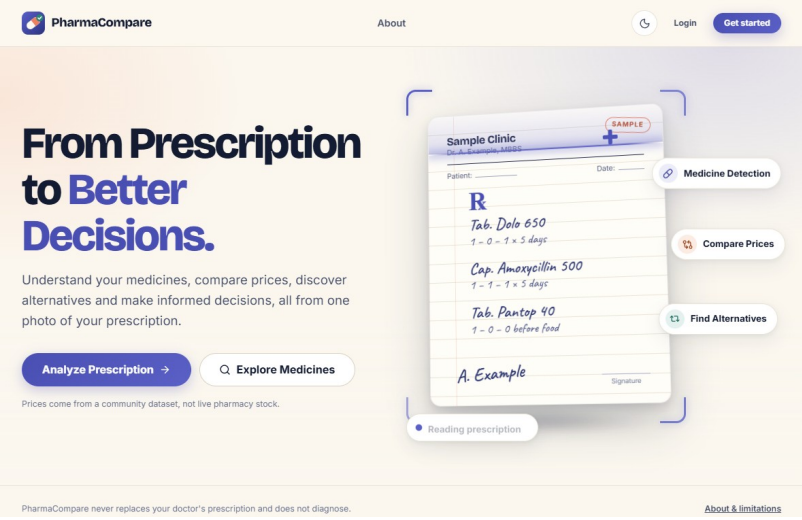


Figure 4.2 Landing page

Figure 4.2 represents the landing page, with the Analyze Prescription and Explore Medicines buttons and the Login and Get started buttons in the header. It states that prices come from a community dataset, not from live pharmacy stock.

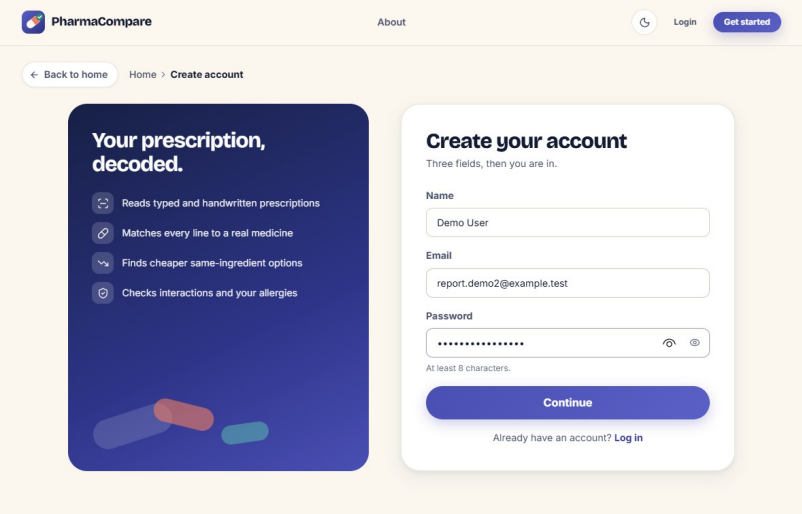


Figure 4.3 Registration form

Figure 4.3 represents the registration form, which asks for a name, an e-mail address and a password of at least eight characters. After Continue the user is taken to the login page.

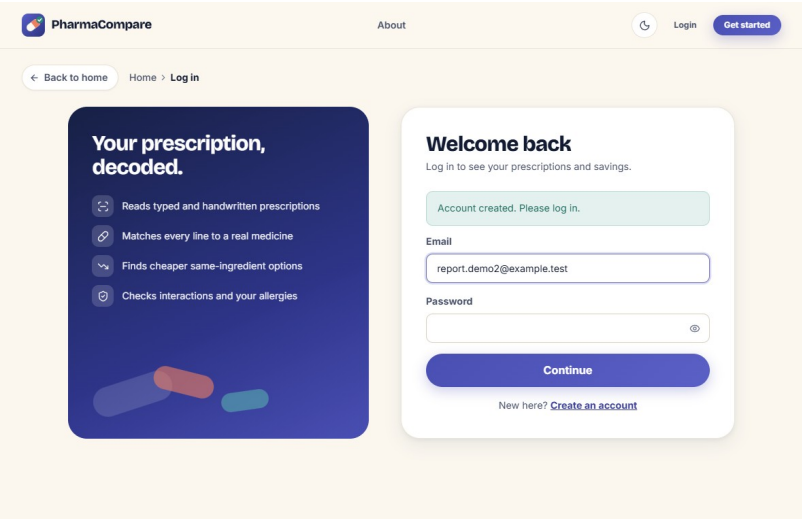


Figure 4.4 Login form

Figure 4.4 represents the login form (e-mail and password). A new user is then taken to a health-preferences page, where drug allergies can be declared or skipped; the list can be edited later on the Profile page.

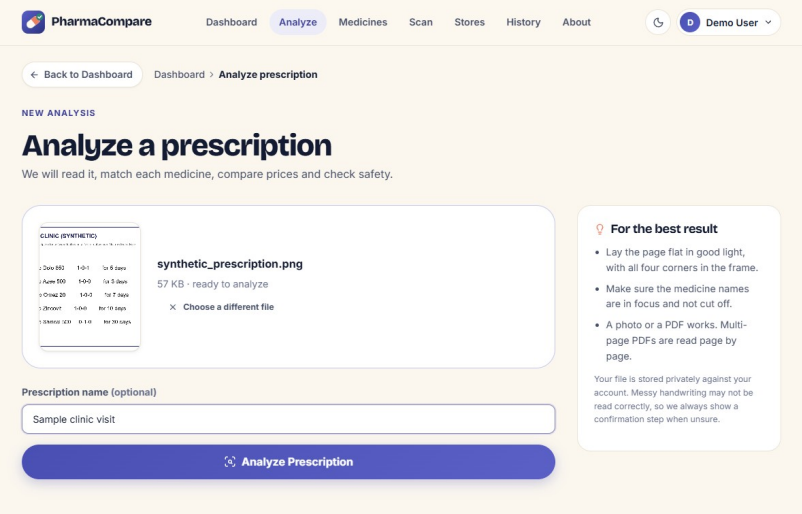


Figure 4.5 Upload page

Figure 4.5 represents the upload page. The user drops or browses for a JPG, PNG or PDF file of up to 10 MB, sees a preview, may name the prescription and presses Analyze Prescription. A five-stage journey is shown while the analysis runs.

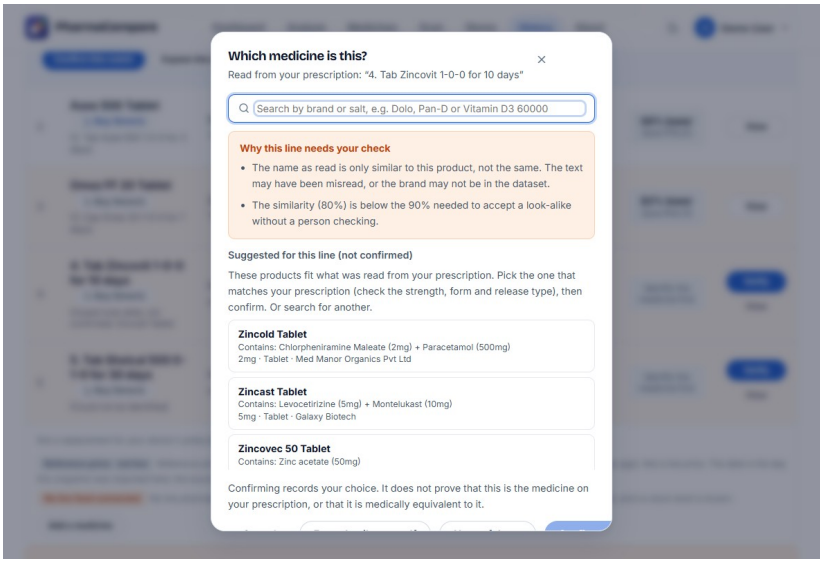


Figure 4.6 Verification dialog

Figure 4.6 represents the dialog opened by Verify. It explains why the line needs a check, lists the look-alikes marked as not confirmed and offers a search box, Enter details yourself and None of these.

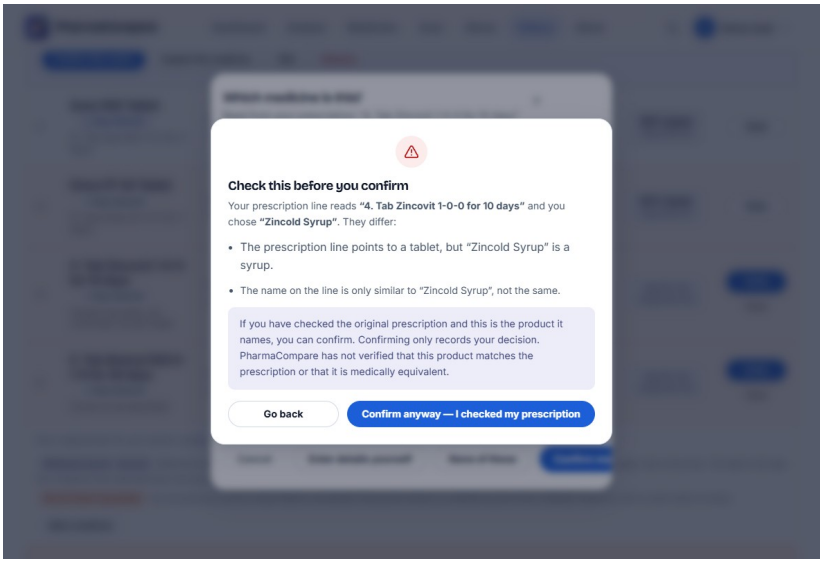


Figure 4.7 Warning before a conflicting confirmation

Figure 4.7 represents the warning shown when the chosen product differs from the line (here a syrup chosen for a line that points to a tablet). The choice is saved only after "Confirm anyway", and the text says that confirming does not show that the product matches the prescription.

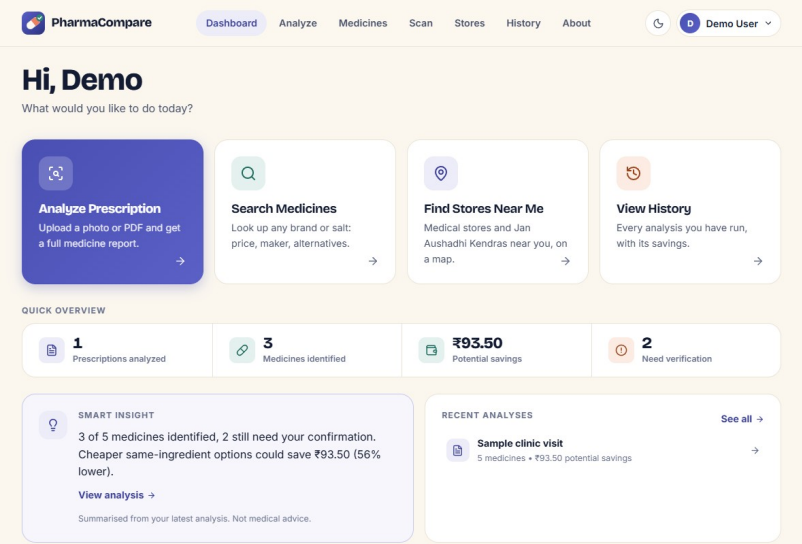


Figure 4.8 Dashboard

Figure 4.8 represents the dashboard: quick actions, a quick overview (prescriptions analyzed, medicines identified, potential savings, lines needing verification), a smart insight and the recent analyses.

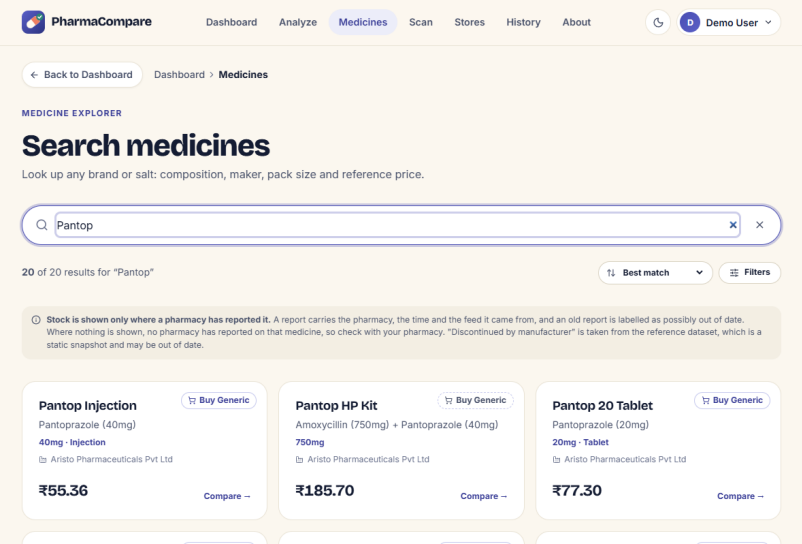


Figure 4.9 Medicine search page

Figure 4.9 represents the medicine search page. A search by brand or salt lists the matching products with composition, strength, form, maker and reference price. A notice explains that stock is shown only when a pharmacy has reported it.

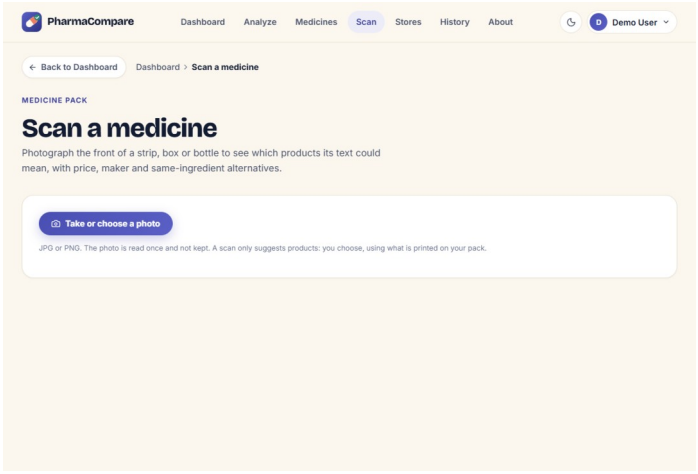


Figure 4.10 Scan a medicine page

Figure 4.10 represents the scan page, where the user takes or chooses a photo of a strip, box or bottle. The photo is read once and not kept, and a scan only suggests products.

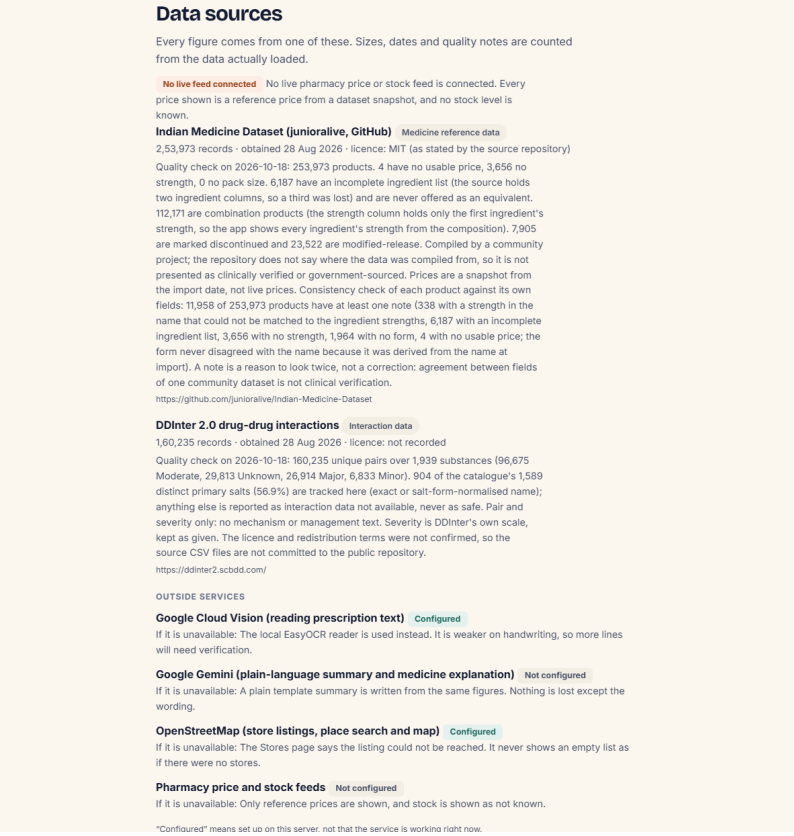


Figure 4.11 Data sources on the About page

Figure 4.11 represents the data sources on the About page: the two datasets with their licence, record count, date obtained and quality notes, the notice that no live feed is connected, and the status of the outside services (configured or not).

Design decisions:

- A **guided journey** (uploading, reading, matching, safety, report ready) is shown during analysis.
- Status words are plain (*Matched*, *Needs verification*) and every price is labelled as a reference-dataset figure, **not a live price**.
- A choice that differs from the prescription is shown with its differences and needs an explicit confirmation.
- The pages state that **stock availability is not tracked** unless a pharmacy has reported it.
- A light and a dark theme are provided, and the layout adapts to narrow screens.

CHAPTER 5

TESTING

The system was tested at the level of single functions, of modules, of complete functions and of the integrated system. All tests are automated: 592 back-end tests (Python unittest) and 186 front-end logic tests (Node test runner). Most back-end tests use the real medicine dataset and the real database, so they exercise the same data that the application uses.

5.1 TESTING OBJECTIVES

The objectives of testing PharmaCompare are:

1. **Correct Behaviour**: to verify that every module works as designed on normal, boundary and invalid input.
2. **Safe Design Rules**: to confirm that an uncertain reading is never analysed as a fact, that a missing interaction record is never reported as safe and that no price is presented as live.
3. **End-to-End Workflow**: to confirm that the complete flow from upload to report works with the real dataset and database.
4. **Data Protection**: to confirm that one user cannot reach another user's prescription and that API keys never appear in logs or responses.

5.2 SCOPE

The testing was planned to:

- Verify that the helper functions (release type, composition, pack size, price rules, query parsing, dosing instructions, photo-quality checks) behave correctly on normal, boundary and invalid input;
- Verify that a line that is not confirmed never reaches the safety, price or recommendation stages;
- Validate that the matcher does not settle a line on a look-alike product, a different strength or a different release type;
- Check that an unreadable image is rejected rather than guessed, and that uncertain OCR lines are marked;
- Check that a choice that differs from the prescription needs an acknowledgement and is logged;

- Check that one user can never read, confirm or delete another user's prescription;
- Check that API keys never appear in logs, error messages or API responses;
- Confirm that the screen and the PDF report show the same statuses and savings.

Scope includes:

- Unit testing
- Module testing
- Functional testing
- System testing

Test environment. The back-end tests were run with Python's *unittest* discovery and the front-end logic tests with the Node test runner. The OCR tests render images locally and read them with EasyOCR, so they make no cloud calls; the secret-safety tests use a fake key and a local fake server; the Gemini path is tested with a mocked answer.

What the tests do not show. They check software behaviour. They do not measure OCR accuracy, medicine-matching accuracy on real prescriptions, or the clinical validity of any message.

5.3 UNIT TESTING

Each helper function was tested individually with normal, boundary and invalid input.

Table 5.1 Unit tests

Unit	Test input	Expected output	Actual output	Result
Release type	Name or pack text with SR, ER, CR markers; plain tablet; "OD"; liquid name; conflicting markers	Marker found; conventional gives none; "OD" and liquid markers ignored; conflict flagged modified	As expected	Pass
Composition	"Paracetamol (250mg/5ml)"; combinations; names with three strengths	Ingredients and strengths split; syrup volume not a dose; incomplete list flagged	As expected	Pass
Pack parsing	Strips, bottles, vials, volumes, weights	Counts, volumes and weights kept apart; same quantity needs same amount and kind	As expected	Pass

Unit	Test input	Expected output	Actual output	Result
Extreme-price rule	Groups of fewer than 8; prices near and beyond 10 times the median; zero price	Small groups never judged; flag only beyond 10 times the median; zero is not a unit price	As expected	Pass
Savings eligibility	Cheaper same-pack alternative; each disqualifier in turn	Clean alternative counts; each disqualified alternative does not	As expected	Pass
Dosing instructions	“1-0-1”; “100” run together; durations as handwritten; a strength or a number with a unit	Written schedule read as written; run-together digits only possibly a schedule; a strength never a schedule; nothing invented for text without dosing	As expected	Pass
Photo-quality checks	Clean page; blurred, dark, faint and small versions of it; a file that is not an image	No warning for the clean page; each defect named in plain words; nothing claimed for a non-image	As expected	Pass
Display rounding	A confidence of exactly 92.5%	Rounded half up (93%) on screen and in the PDF	As expected	Pass
Pharmacy feed rows	Missing or future time; zero price; unknown stock word; ambiguous product name	Each row rejected with its reason; an ambiguous name is never chosen	As expected	Pass
Salt-name normalisation	Spelling variants and salt forms; unrelated names	Variants normalised; unrelated names untouched; every alias target exists in DDInter	As expected	Pass
Query parsing	“Pan D”, “Pan-D”, “T. pan-D”; a strength written next to its unit	Same query for every spelling; leading T is a form hint; strength glued to its unit	As expected	Pass
Match gates	Line stating a strength against a product with none; one-ingredient line against a combination	A product with no strength on record cannot settle the line; a combination cannot settle a one-ingredient line	As expected	Pass
Secret redaction	Messages containing the API key in several forms	Key removed in every form; ordinary diagnostics kept	As expected	Pass

Observation:

All unit tests passed. The helper functions handle boundary cases such as groups that are too small to judge, packs of different kinds, names that carry several strengths and schedules that handwriting turns into other digits.

5.4 MODULE TESTING

The system was divided into modules (OCR, matching, dosing and timeline, verification, scan, safety, price and alternatives, pharmacy reports, report, routes and database scripts). Each module was tested for correctness before the modules were combined.

Table 5.2 Module tests

Module	Test description	Expected output	Actual output	Result
OCR	Clean printed prescription image; degraded or unreadable image	Every medicine is found; no wrong medicine is produced; an unreadable image is rejected, not guessed	As expected	Pass
OCR	Provider selection when the cloud service fails or is pinned; request languages	Fallback to EasyOCR is logged and flagged; a pinned cloud provider never falls back; English and Hindi requested	As expected	Pass
Matching	Known brands; under-specified lines; SR line against conventional products	Known brands match; under-specified lines get options; an SR line never matches a conventional product	As expected	Pass
Matching	Headings, durations and instructions of a handwritten prescription	None is matched to a product; instructions and Hindi notes are listed as notes	As expected	Pass
Verification	A look-alike product chosen for a line; a different strength, form, release type or ingredient	The differences are returned; nothing is saved until they are acknowledged; the choice never counts in savings	As expected	Pass
Verification	“None of these”; details typed by the user; bad input	Status user_unmatched; typed medicine kept as entered by the user, never matched or priced; bad input refused	As expected	Pass
Audit trail	Each decision on a line	A System_Logs row records the line as read, its status, the candidates and the warnings; the line text is unchanged	As expected	Pass
Timeline	Identified, unidentified and unread lines	Only identified medicines appear by name; others are listed apart; a possible schedule is marked possible	As expected	Pass

Module	Test description	Expected output	Actual output	Result
Medicine scan	A pack photo; text that resembles nothing; an OCR failure	Candidates only, never a confirmation; no candidate is invented; the photo is deleted in every case	As expected	Pass
Safety	No confirmed medicine; user with and without declared allergies	“No check ran” message; honest “unchecked” allergy message; owner’s allergies used	As expected	Pass
Price and alternatives	Alternative in a different pack; zero-priced alternative; extreme price; incomplete composition	Different pack not counted; zero price never offered; extreme price flagged and kept; incomplete products never offered	As expected	Pass
Pharmacy reports	No reports; fresh and stale stock; the same feed imported twice; a bad partner key	“Not reported”, never “in stock”; stale stock not counted; nothing added twice; one 401 for every bad key	As expected	Pass
Price provenance	Reference price; no feed connected	Source, import date and “not a live price” stated; “no live feed connected” stated	As expected	Pass
Report summary	Faithful, invented and advice-giving summaries; no key; failing Gemini call	A faithful summary is accepted; an invented number or advice is rejected; the local summary replaces it	As expected	Pass
Report (PDF)	Hostile text in a line; every detected line; page numbers	Generation survives; every line has a row; footer numbers every page	As expected	Pass
Routes	Confirm, add, remove, search; another user’s line	Exactly the chosen medicine is saved; other accounts are refused	As expected	Pass
Database scripts	Migration run twice; audit script; fresh install from schema	Migration is idempotent; audit is read-only; fresh schema equals the live one	As expected	Pass
Secret safety	Network failure, error body echoing the key, debug logging (cloud OCR and Gemini)	No key in any message, traceback, log or API response	As expected	Pass

Observation:

All modules functioned independently and produced the expected output. The module tests also confirm the design rule that an uncertain line is never settled by the OCR confidence or the matcher alone.

5.5 FUNCTIONAL TESTING

Functional testing validates the end-to-end behaviour of a function from the user’s input to the system’s output, with the real database.

Table 5.3 Functional tests

Test case	Input condition	Expected output	Actual output	Status
Printed prescription	Image with several medicines	Every medicine found and listed	Found	Pass
Handwritten prescription	Real captured OCR output of a handwritten page, with a Hindi note	Five medicine lines; the note is a note, not a medicine; nothing unconfirmed is priced or safety checked	As expected	Pass
Uncertain OCR line	One line read with low confidence	Only that line flagged “check match”; no saving for it; API and PDF show the same status	As expected	Pass
Look-alike brand	A brand that is not in the dataset	Closest look-alikes with reasons, “not confirmed”; never a suggestion presented as fact	As expected	Pass
Conflicting confirmation	A syrup chosen for a line that points to a tablet	Differences shown; saved only after acknowledgement; kept out of the headline savings; logged	As expected	Pass
Typed medicine	User types a medicine the dataset lacks	Kept as entered by the user, with the allergy notice; never matched, priced or checked	As expected	Pass
Generic-name line	Line names an ingredient but no brand	Flagged “brand unspecified”; no price or saving against one arbitrary brand	As expected	Pass
Medicine timeline	A schedule “1-0-1” for a matched medicine	Morning and night entries with the written duration; unidentified lines listed apart	As expected	Pass
Poor photo	A blurred upload	Upload succeeds with warnings that say what to fix; the OCR text is unchanged	As expected	Pass
Medicine scan	Photo of a pack	Possible products with reasons, never a confirmation; the photo is deleted	As expected	Pass
Access control	A second account requests another user’s prescription or report	Request refused	Refused	Pass
Search	Spellings of “Pan-D”; vitamin D3 60000; release word in the query	The intended brand first; ingredient and release type respected; nonsense returns nothing	As expected	Pass
Screen and PDF consistency	One analysis shown on screen and in the PDF	Same lines, status wording, confidence wording and savings totals	As expected	Pass

Observation:

Functional testing confirmed that the main functions (reading, identification, verification, safety screening, price comparison, timeline and reporting) operate as designed, and that the confirmation gate holds at every stage.

5.6 SYSTEM TESTING

System testing validates the integration of all modules (OCR, matching, safety, prices, report, API and front-end logic) as one unit. The complete suites were run on the final version of the system.

Table 5.4 Back-end test suite by group

Test file	Focus	Tests	Result
test_matching	Matcher behaviour and alternatives	26	Pass
test_matching_safety	Gates against look-alike, incomplete and mismatched products	27	Pass
test_picker_and_confirmation	Search ranking and the confirmation routes	37	Pass
test_verification_workflow	Verification workflow: reasons, conflict warnings, audit trail, typed details	44	Pass
2 files	Handwritten-style prescriptions end to end, with real captured OCR output	34	Pass
2 files	Generic-name lines, candidates and same-ingredient alternatives	22	Pass
2 files	Local OCR, unreadable images, uncertain lines, OCR logging	25	Pass
test_image_quality	Photo-quality checks	16	Pass
test_instructions_timeline	Dosing instructions, notes and the medicine timeline	41	Pass
test_scan	Medicine scan	14	Pass
test_report_summary	Report summary and Gemini fallback	20	Pass
test_interactions_allergy_pdf	Salt names, allergy ownership, PDF report	15	Pass
2 files	Pack parsing, extreme-price rule, savings, release type, composition	27	Pass
2 files	Pharmacy reports, price provenance and the partner feed	68	Pass
2 files	Data sources and data validation	32	Pass
2 files	No key in messages or logs (cloud OCR and Gemini)	27	Pass
2 files	Migration, audit script and fresh schema	6	Pass
2 files	Independent location feature (not part of the prescription workflow)	111	Pass
Total		**592**	**All passed (378 s)**

Table 5.5 Front-end logic test suite by group

Test file	Focus	Tests	Result
analysis	Screen and PDF show the same lines, status, confidence and savings	16	Pass
2 files	Verification dialog, warnings and the “none of these” choice	30	Pass
handwriting	Handwritten-style prescription on screen	14	Pass
4 files	No simulated or live-stock wording; pharmacy reports and sources	36	Pass

Test file	Focus	Tests	Result
4 files	Medicine timeline, photo notes, summary and alternatives notes	21	Pass
2 files	Medicine scan and the Buy Generic button	31	Pass
2 files	Independent location feature (not part of the prescription workflow)	38	Pass
Total		**186**	**All passed**

Observation:

- All 778 tests passed (592 back-end and 186 front-end), and none was skipped.
- The suite protects the design rules: an unconfirmed line gets no analysis, price or saving, and the screen and the PDF agree. It checks software behaviour, not OCR accuracy or clinical completeness.

CHAPTER 6

RESULTS

This chapter presents **snapshots of the working system**, the reference data it runs on, two dataset-level experiments and the status of each objective.

6.1 SNAPSHOTS OF THE PROJECT

To avoid real patient data, a **fictional printed prescription** (Figure 6.1) was uploaded with a test account. It has five medicine lines with schedules and durations; one brand (Omez, written as a capsule) exists in the dataset only as a tablet, and two brands (Zincovit, Shelcal) are not in the dataset. The screens below are the system's real outputs on this one image; they are a demonstration, **not an accuracy measurement**.

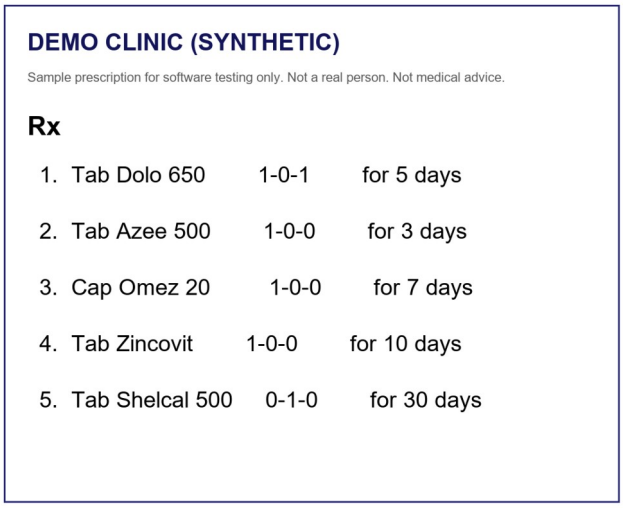


Figure 6.1 Fictional demonstration prescription

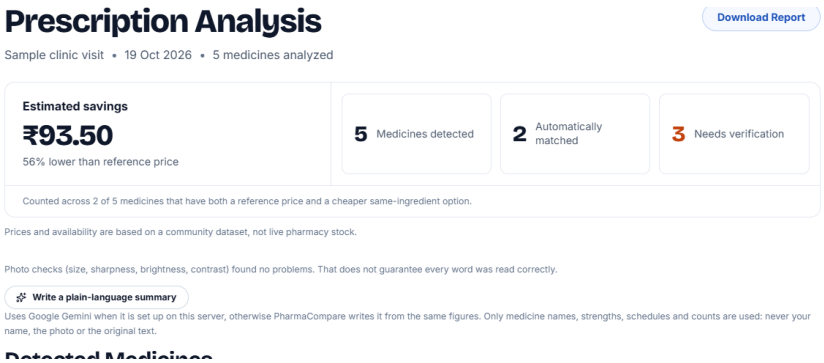


Figure 6.2 Analysis summary

Analysis result. Figure 6.2 shows the summary: five medicines detected, **two matched automatically** and **three needing verification**, with savings counted across two of the five medicines.

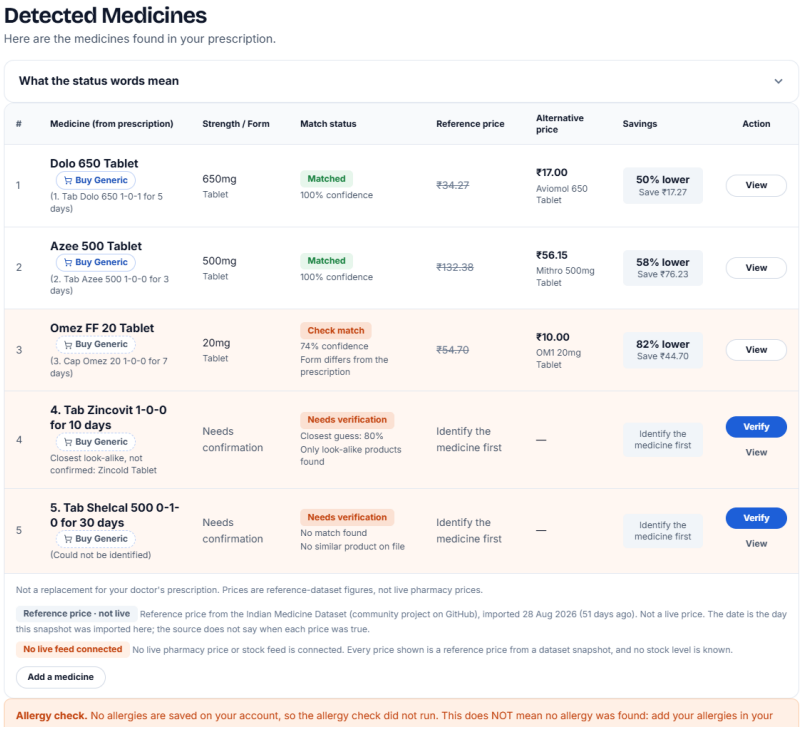


Figure 6.3 Detected medicines

Figure 6.3 shows the detected medicines. The two matched lines are priced and show a cheaper same-ingredient option. The Omez line is **Check match**: the prescription says capsule, the product is a tablet. The two lines that need verification show their closest look-alike, or that nothing similar exists, and **no price**.

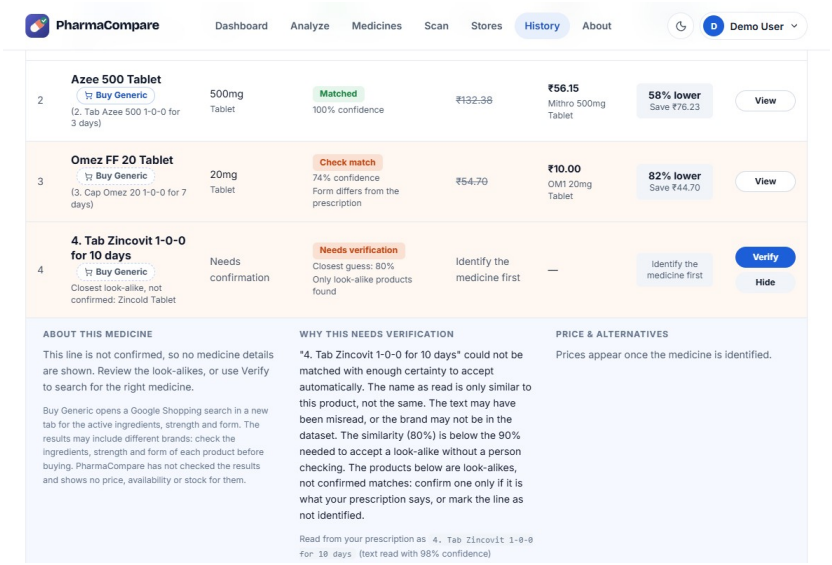


Figure 6.4 Details of a line that needs verification

Figure 6.4 explains in words why Zincovit was not accepted: the name is only similar to Zincold Tablet, and the similarity of 80% is below the 90% needed to accept a look-alike.

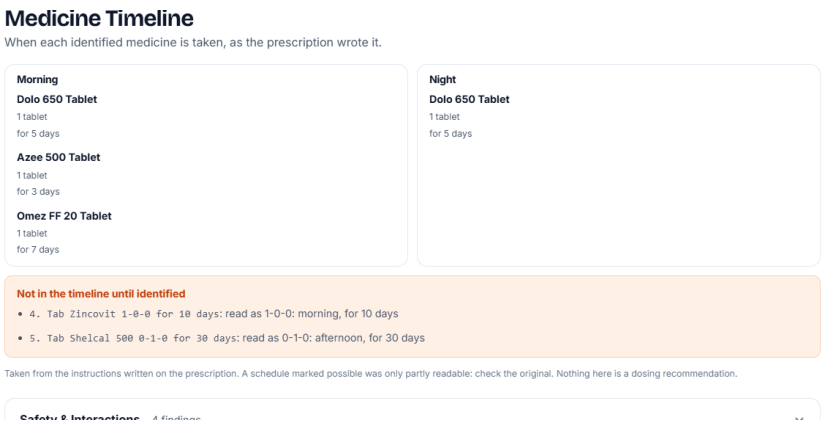


Figure 6.5 Medicine timeline

Figure 6.5 shows the timeline. Dolo 650 appears in the morning and at night for five days, Azee 500 and Omez in the morning; the two lines that are not identified are listed apart with the schedule read from them.

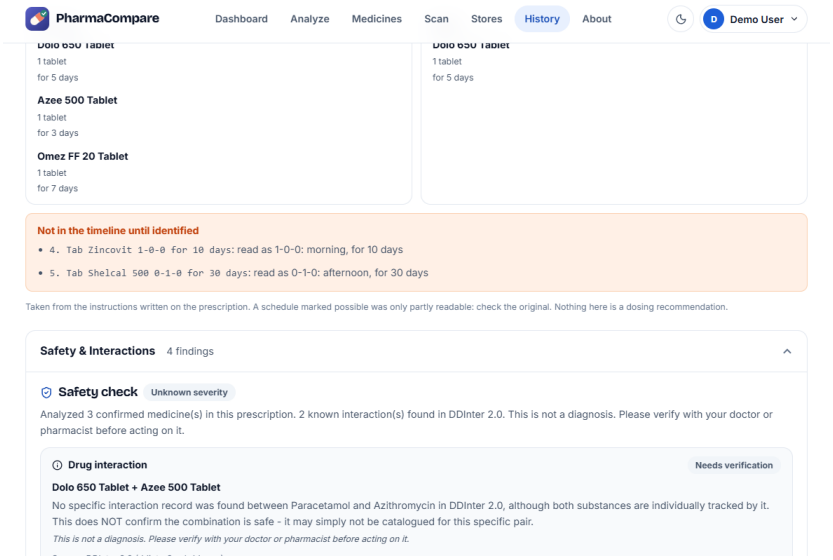


Figure 6.6 Safety and interactions

Figure 6.6 shows the safety panel. DDInter records two interactions among the three confirmed medicines, both with severity Unknown; the pair paracetamol with azithromycin has no record and is shown as **needs verification**, never as safe, and the allergy check says that it did not run because no allergy is saved.

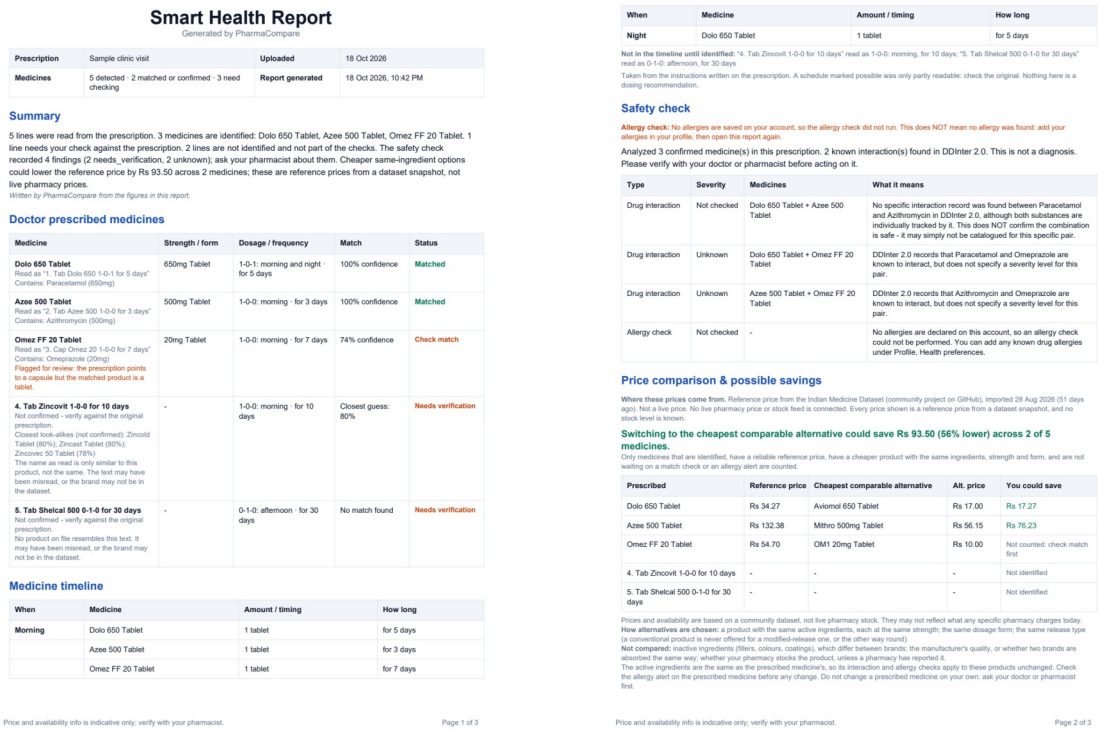


Figure 6.7 Smart Health Report (pages 1 and 2)

Figure 6.7 shows the Smart Health Report: a summary, the medicines with their dosing and status, the timeline, the safety findings and the savings, with the same wording as the screen.

6.2 OBSERVATIONS

6.2.1 REFERENCE DATA USED BY THE SYSTEM

Table 6.1 Reference data used by the system

Item	Value
Medicine products in the table	253,973 (imported 28 August 2026 from a community-compiled dataset of unstated origin)
Discontinued products	7,905
Distinct primary ingredients; distinct manufacturers	1,589; 7,648
Modified-release products	23,522
Products with an incomplete ingredient list	6,187 (2.4%)
Products with a blank or “NA” strength	3,656 (1.4%)
Exact duplicate records	24 surplus rows (19 groups carry different prices)
Interaction pairs from DDInter 2.0	222,383 raw rows reduced to 160,235 unique pairs over 1,939 names
Severity of the pairs	Major 26,914; Moderate 96,675; Minor 6,833; Unknown 29,813
Ingredients with a DDInter match	about 56% of the distinct ingredients
Products with at least one validation note	12,007 (checked against their own fields; a reason to check, not a correction)
Registered data sources; connected pharmacy feeds	2 (medicine dataset, DDInter 2.0); none

The DDInter 2.0 paper reports **302,516 records for 2,310 drugs** [19]; the downloadable bulk files gave fewer pairs and names, and the reason is **not established**, so only this project’s own counts are given. The medicine data contain duplicates, blank strengths and implausible prices, which is why suspect rows are flagged, not deleted.

6.2.2 EXPERIMENT A - INGREDIENT-ONLY VERSUS COMPARABLE-PRODUCT RULE

500 products were drawn at random (seed 20261016) from the 239,995 live, priced products with a complete ingredient list. A deliberately simple baseline looked for the cheapest

product with the **same primary ingredient**, whatever its strength, form or pack; PharmaCompare's own comparison ran on the same product. The baseline is not a model of any published system.

Table 6.2 Ingredient-only versus comparable rule

Measure	Result
Products for which the baseline offered a cheaper item	499 of 500
Of these, cheapest item rejected by the comparability and price-quality rules	460 (92.2%)
Median apparent saving of the rejected suggestions	91.4%
Products for which at least one comparable alternative was listed	481 of 500
Products with at least one countable saving	444 of 500

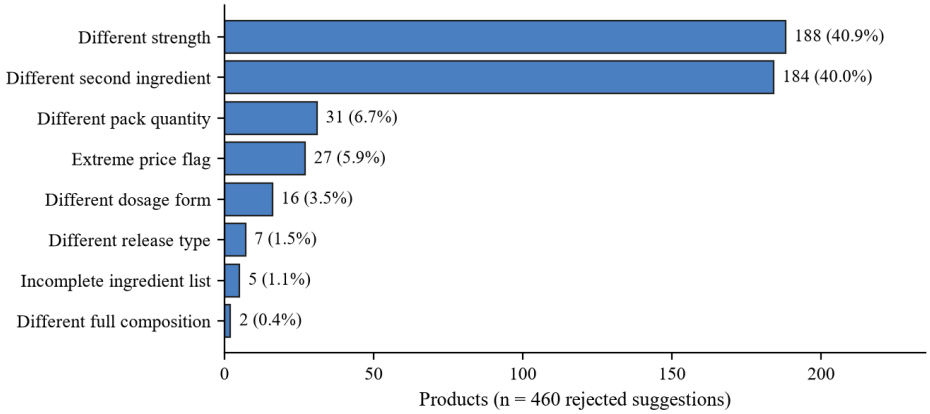


Figure 6.8 Why ingredient-only suggestions fail

In four cases out of five (372 of 460) the first rule to fail was a **different strength or second ingredient**. The 91.4% median apparent saving of the rejected suggestions reflects products of a different strength or composition, **not a real saving**.

6.2.3 EXPERIMENT B - SYNTHETIC NOISE TEST OF THE MATCHER

With no labelled real prescriptions available, **300 product names** (8–40 characters, seed 20261017) were submitted to the matcher unchanged and with **one, two or three simulated character edits** (confusable substitutions such as o↔0, deletions, duplications). The true product is always in the database and no strength text is present. The probe tests **how cautious the matcher is**, not OCR accuracy.

Table 6.3 Matcher behaviour under simulated noise

Edits per line	Auto-accepted	Auto-accepted wrongly	True product reachable by the user
0 (clean)	285 (95.0%)	0	290 (96.7%)
1	106 (35.3%)	3 of 106 (2.8%)	225 (75.0%)
2	26 (8.7%)	0	154 (51.3%)
3	9 (3.0%)	0	115 (38.3%)

“Auto-accepted” means matched and not left to a choice. “Reachable” means that the product was auto-accepted correctly, or was among the options or the closest guess shown to the user.

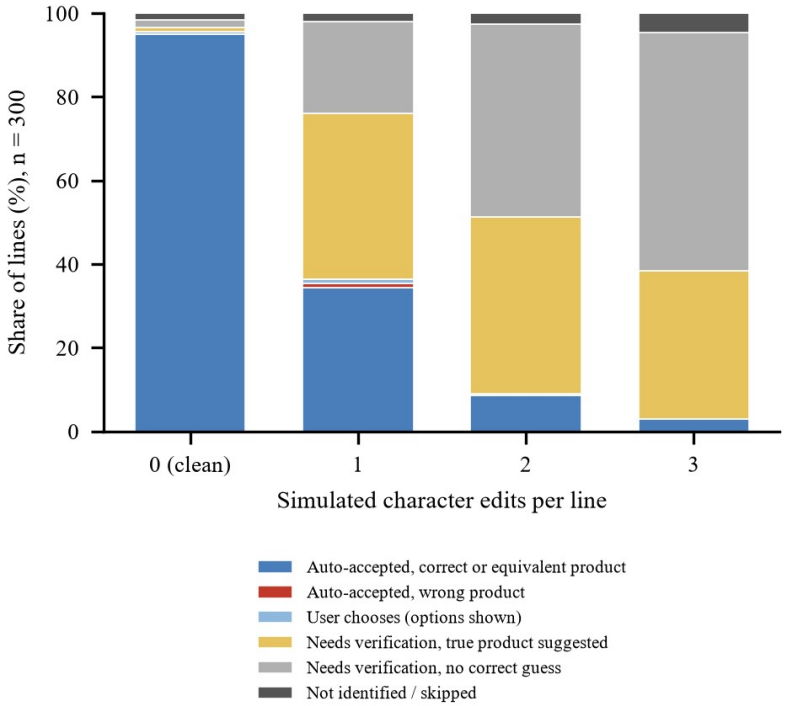


Figure 6.9 Matcher outcomes by simulated edits

With clean names **95.0%** of lines were auto-accepted and none wrongly. With one edit **35.3%** were auto-accepted and **3 of 106 (2.8%)** wrongly (95% Wilson interval 1.0–8.0%); with two and three edits 8.7% and 3.0% were auto-accepted, none wrongly. The wrong acceptances show where name matching reaches its limit: a dropped letter turned “Azicof Syrup” into the exact name of a different product, “Aicof Syrup”, and “Rosuline-F 20” became “Rosuline 20”.

6.2.4 STATUS OF THE OBJECTIVES

Table 6.4 Status of the objectives

Objective	Status	What the system does	Main limitation
1. Real-time price comparison	Partly implemented	Compares products with the same composition, strength, form, release type and pack quantity, per unit; every price states its source, import date and that it is not live; pharmacy-reported prices are shown separately	Reference snapshot of 28 August 2026; no live price source is connected
2. AI-based prescription analysis	Implemented, rule-based	Reads and matches the lines, reads the dosing, and checks interactions, duplicates, strengths and allergies on confirmed lines	No trained model; limited to DDInter coverage; accuracy on real prescriptions not measured
3. Alternative drug recommendation	Implemented	Up to three cheaper products with the same ingredients, strength, form and release type, with notes on what was not compared; never for an unconfirmed line	No therapeutic-equivalence or bioequivalence data
4. Real-time availability tracking	Partly implemented	Receives and shows pharmacy-reported stock with its age; otherwise says that stock is not tracked	No pharmacy sends reports, so no stock level is known
5. Confidence-based decision support	Implemented, heuristic	Status with reasons, a similarity score, conflict warnings and a recorded decision trail gate every later stage	Scores are heuristics, not calibrated probabilities
6. Use of healthcare data sources	Partly implemented	Two registered sources with licence, date and measured quality notes; product-level validation	Neither source is authoritative; no official register is integrated

6.2.5 DISCUSSION

The experiments support the design but do not prove an outcome. The matcher is **cautious**: after a single character error about 65% of lines go to the user, so the verification dialog matters, and its usability has not been studied. The wrong acceptances occur where a different real product lies one edit away from the true name, which no similarity score can resolve. Comparing on ingredient alone would often propose a product that is **not a substitute**.

CHAPTER 7

CONCLUSION

7.1 CONCLUSION

PharmaCompare turns a prescription image into a list of **medicines the user has confirmed**, the **dosing written on the prescription**, **rule-based safety messages** and **comparable-product prices**, and it refuses to analyse a line it cannot settle. The system is rule-based and database-driven; its prices come from a **static reference dataset**, and no live price or stock feed is connected.

Text is read with Google Cloud Vision (English and Hindi) or EasyOCR and matched against **253,973 medicine products**; only strong, corroborated matches are accepted automatically, and every other line waits for the user, who sees the reasons, is warned about a choice that differs from the prescription, and leaves an audit trail. Confirmed medicines are screened against **160,235 interaction pairs** and priced against products of the same composition, strength, form, release type and pack quantity. The result is shown on screen, with a medicine timeline, and as a Smart Health Report, and every output states its source and its limits.

All 592 back-end and 186 front-end tests passed. The comparability rule rejected **460 of 499** ingredient-only price suggestions, and the matcher referred most noisy lines to the user, at the cost of 3 wrong acceptances in 106 auto-accepted lines at one simulated edit.

The prototype meets the main intended outcomes:

- Reading, identification, safety screening, price comparison and a medicine timeline in one workflow;
- Uncertain readings handed to the user, with reasons and warnings, instead of being presented as facts;
- Strict comparable alternatives and honest outputs (“needs verification”, “not live”, “not tracked”).

An evaluation on labelled real prescriptions is still missing: handwriting accuracy was not measured on a labelled set, so no OCR or matching accuracy on real prescriptions is reported, and no clinical validation, user study or latency measurement was done.

7.2 FUTURE SCOPE

The system has the potential for the following enhancements:

1. **Evaluation on Real Prescriptions**

   Collect consented, anonymised prescription pages with pharmacist-verified labels, tune the thresholds on a development split and report matching outcomes with confidence intervals.

2. **Live Prices and Availability**

   Connect a pharmacy or distributor to the existing feed receiver, or integrate licensed or official price and stock sources, so that prices and availability can be live and per pharmacy.

3. **Official Data Sources**

   Add authoritative registers when their terms allow, so that the medicine data no longer depend on a community dataset.

4. **Treatment-Course Cost**

   Add the cost of a full course and therapeutic-equivalence information, using the schedules and durations that are already read.

5. **Regional Languages and Handwriting**

   Support more regional languages and test on real handwriting.

6. **Usability and Secure Deployment**

   Study the usability of the verification flow and add privacy controls and secure hosting before any real use.

APPENDIX-A

LIST OF ABBREVIATIONS

The abbreviations used in this report are listed in the table below.

Sl. No.	Abbreviation	Full form
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	BiLSTM	Bidirectional Long Short-Term Memory
4	CNN	Convolutional Neural Network
5	CORS	Cross-Origin Resource Sharing
6	CR	Controlled Release
7	CRNN	Convolutional Recurrent Neural Network
8	DDInter	Drug–Drug Interaction database
9	DFD	Data Flow Diagram
10	ER	Entity–Relationship (diagram); Extended Release (release type)
11	FK	Foreign Key
12	IEEE	Institute of Electrical and Electronics Engineers
13	JSON	JavaScript Object Notation
14	JSX	JavaScript XML
15	JWT	JSON Web Token
16	LLM	Large Language Model
17	MIT	Massachusetts Institute of Technology (software licence)
18	NLP	Natural Language Processing
19	NSAID	Non-Steroidal Anti-Inflammatory Drug
20	OCR	Optical Character Recognition
21	PDF	Portable Document Format
22	PK	Primary Key
23	RAM	Random Access Memory
24	REST	Representational State Transfer
25	RNN	Recurrent Neural Network
26	SDLC	Software Development Life Cycle
27	SR	Sustained Release
28	SRS	Software Requirement Specification
29	TrOCR	Transformer-based Optical Character Recognition
30	YOLO	You Only Look Once (object detector)

APPENDIX-B

HARDWARE AND SOFTWARE DESCRIPTION

The hardware and the software used to build, run and test PharmaCompare are described below.

Hardware description

1. **Computer** – a processor with six cores and 16 GB RAM: runs the MySQL server, the back end and the OCR model.
2. **Camera or scanner** – used by the user to capture the prescription or a medicine pack as a JPG, PNG or PDF file.
3. **Network connection** – needed only by the optional cloud services (Google Cloud Vision and Gemini).

Software description

1. **Python and Flask** – language and web framework of the REST API and the services.
2. **MySQL 8.0** – relational database holding the sixteen tables, with a full-text index for medicine search.
3. **EasyOCR / Google Cloud Vision** – local and cloud OCR engines used to read the prescription and the pack photo.
4. **React and Vite** – single-page front end and its build tool; **Axios** for API calls.
5. **ReportLab** – generates the Smart Health Report in PDF.
6. **DDInter 2.0** – open drug–drug interaction database used for the safety screening.
7. **Community medicine dataset (MIT licence)** – 253,973 Indian medicine products used for matching and price comparison; not an official or verified source.
8. **Google Gemini (optional)** – words the report summary and medicine explanations from stored facts.
9. **Unittest and the Node test runner** – automated back-end and front-end tests.

REFERENCES

[1]. Brits, H., Botha, A., Niksch, L., Terblanché, R., Venter, K. and Joubert, G. (2017) Illegible handwriting and other prescription errors on prescriptions at National District Hospital, Bloemfontein. South African Family Practice, 59(2), 50. https://doi.org/10.4102/safp.v59i1.4658

[2]. Ray, A., Najmi, A., Khandelwal, G. and Sadasivam, B. (2020) A cost variation analysis of drugs available in the Indian market for the management of thromboembolic disorders. Cureus, 12(5), e7964. https://doi.org/10.7759/cureus.7964

[3]. Behera, D.K., Rahut, D.B., Mehboobali, M.D., Tasneem, S.H. and Rajendhran, A. (2025) Medicine affordability and access in India: lessons from generic–branded price variation under the Jan Aushadhi Scheme. Frontiers in Public Health, 13. https://doi.org/10.3389/fpubh.2025.1629835

[4]. Razdan, A., Raghavan, A., Panandikar, M., Pol, A., Hedaoo, K. and Patil, R. (2023) Recognition of Handwritten Medical Prescription using CNN Bi-LSTM With Lexicon Search. 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), 1–6. https://doi.org/10.1109/ICCCNT56998.2023.10307451

[5]. Uz Zaman, M.F. et al. (2024) Recognizing Medicine Names from Bangladeshi Handwritten Prescription Images using TrOCR. 2024 7th Asia Conference on Cognitive Engineering and Intelligent Interaction (CEII), 55–59. https://doi.org/10.1109/CEII65291.2024.00020

[6]. Ali, U., Ranmbail, S., Nadeem, M., Ishfaq, H., Ramzan, M.U. and Ali, W. (2024) Leveraging Deep Learning With Multi-Head Attention for Accurate Extraction of Medicine From Handwritten Prescriptions. 2024 International Conference on Robotics and Automation in Industry (ICRAI), 1–6. https://doi.org/10.1109/ICRAI62391.2024.10894218

[7]. Roy, A., Saha, D. and Debnath, S. (2026) A YOLO-TrOCR Pipeline with Dictionary Refinement for Handwritten Medicine Name Extraction. 2026 International Conference on Power, Electronics, Communications, Computing, and Intelligent Infrastructure (PECCII), 1–6. https://doi.org/10.1109/PECCII70991.2026.11662060

[8]. Devi, M.T.S. and Sudhindra, J. (2026) GENERIC RX: AI-Powered Prescription OCR for Generic Medicine Discovery. 2026 Third International Conference on Innovations in Cybersecurity and Data Science (ICICDS), 1546–1552. https://doi.org/10.1109/ICICDS70526.2026.11604829

[9]. Thota, S. and Shirisha, K. (2026) AI-Powered Prescription Decoder and Generic Medicine Recommender. International Journal on Science and Technology, 17(2). https://doi.org/10.71097/ijsat.v17.i2.10892

[10]. Diaz Amaya, E., Solier Quispe, E.C. and Hijar Flores, J.K. (2024) Information System for price comparison and prescription reading using automation technologies and digitalization of texts for pharmacies in Peru. Proceedings of the 22nd LACCEI International Multi-Conference for Engineering, Education and Technology (LACCEI 2024). https://doi.org/10.18687/laccei2024.1.1.1548

[11]. Jeevidha, S., Yogeshwaran, P., Kannathasan, K. and Anand, A.P. (2026) An Advanced Electronic Health Record for Prescription Analysis Using YOLO and OCR with Spell Correction. 2026 9th International Conference on Computational Intelligence in Data Science (ICCIDS), 1–6. https://doi.org/10.1109/ICCIDS69108.2026.11407882

[12]. Mankash, T., Kota, V.S.C., De, A. and Prakash, P. (2024) MIRAGE: Multimodal identification and recognition of annotations in Indian general prescriptions. arXiv preprint arXiv:2410.09729. (Preprint; author list as shown on the arXiv record.) https://doi.org/10.48550/arXiv.2410.09729

[13]. Agrawal, M., Mishra, V., Jogani, H.S., Kashyap, P. and Mishra, R.G. (2023) A Review of Artificial Intelligence in Medical Prescription Analysis. 2023 3rd International Conference on Pervasive Computing and Social Networking (ICPCSN), 35–43. https://doi.org/10.1109/ICPCSN58827.2023.00014

[14]. Ma, R. et al. (2026) An intelligent OCR-LLM prescription review system for inpatient narcotic and psychotropic drugs. Medicine, 105(38), e50741. https://doi.org/10.1097/MD.0000000000050741

[15]. Patil, R., Peshave, P. and Kamble, M. (2022) Application of Fuzzy Matching Algorithms for Doctors Handwriting Recognition. 2022 IEEE Bombay Section Signature Conference (IBSSC), 1–5. https://doi.org/10.1109/IBSSC56953.2022.10037486

[16]. Kim, W.T. et al. (2024) Medication Extraction and Drug Interaction Chatbot: Generative Pretrained Transformer-Powered Chatbot for Drug-Drug Interaction. Mayo Clinic Proceedings: Digital Health, 2(4), 611–619. https://doi.org/10.1016/j.mcpdig.2024.09.001

[17]. Jagatap, A., Merugu, S. and Comar, P.M. (2025) RxLens: Multi-Agent LLM-powered Scan and Order for Pharmacy. Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track), 822–832. https://doi.org/10.18653/v1/2025.naacl-industry.63

[18]. Abdullah, S. and Mamun, K.A. (2026) An offline smartphone check for drug interactions and past reactions in handwritten prescriptions: A technical feasibility study in rural Bangladeshi outpatient care. arXiv preprint arXiv:2608.27239v2. (Preprint, not peer reviewed.) https://doi.org/10.48550/arXiv.2608.27239

[19]. Tian, Y. et al. (2025) DDInter 2.0: An enhanced drug interaction resource with expanded data coverage, new interaction types, and improved user interface. Nucleic Acids Res., 53(D1), D1356–D1362. https://doi.org/10.1093/nar/gkae726

[20]. Alharbi, O.M. (2026) Designing an OCR-Based Medication Support System With Drug–Drug Interaction Alerts for Elderly. IEEE Access, 14, 13688–13701. https://doi.org/10.1109/ACCESS.2026.3651478

[21]. Nasser, F., Hasan, H., Salman, S., Mohanna, A. and Chehab, A. (2026) PharmaBot: A Multimodal AI System for Drug Interaction Detection. 2026 IEEE 5th International Multidisciplinary Conference on Engineering Technology (IMCET), 7–14. https://doi.org/10.1109/IMCET69180.2026.11503766

[22]. Gaikwad, K., Kharche, S., More, R., Kadam, S., Bobade, S. and Patil, P. (2025) OCR AND NER Based Generic Medicine Recommendation System. 2025 International Conference on Intelligent and Secure Engineering Solutions (CISES), 1253–1259. https://doi.org/10.1109/CISES66934.2025.11265361

[23]. Janani, M., Laya, H. and Mathishree, B. (2025) OCR Text Recognition and Recommendation Using Machine Learning. 2025 International Conference on Visual

Analytics and Data Visualization (ICVADV), 804–807. https://doi.org/10.1109/ICVADV63329.2025.10961754

[24]. Karmakar, J. et al. (2025) Optimizing Pharmaceutical Affordability: A Composition-Based Algorithm for Medicine Price and Alternative Discovery. 2025 International Conference on Engineering Innovations and Technologies (ICoEIT), 1194–1198. https://doi.org/10.1109/ICoEIT63558.2025.11211845

[25]. Larson M, J.S., Sivakumar, R. and Murugesan, R. (2026) AI-Enabled Brand-to-Generic Medication Recommendation System for Pharmacy: Addressing Consumer Brand-Name Bias. 2026 International Conference on Communication, Computing and Emerging Technologies (IC3ET), 358–363. https://doi.org/10.1109/IC3ET64989.2026.11467238